



2026

MODUL PRAKTIKUM 3

ARTIFICIAL NEURAL NETWORK

OPTIMIZED VERSION

Disusun Oleh:

Ryan F F Hakim

Dipersembahkan Untuk:

Sains Data

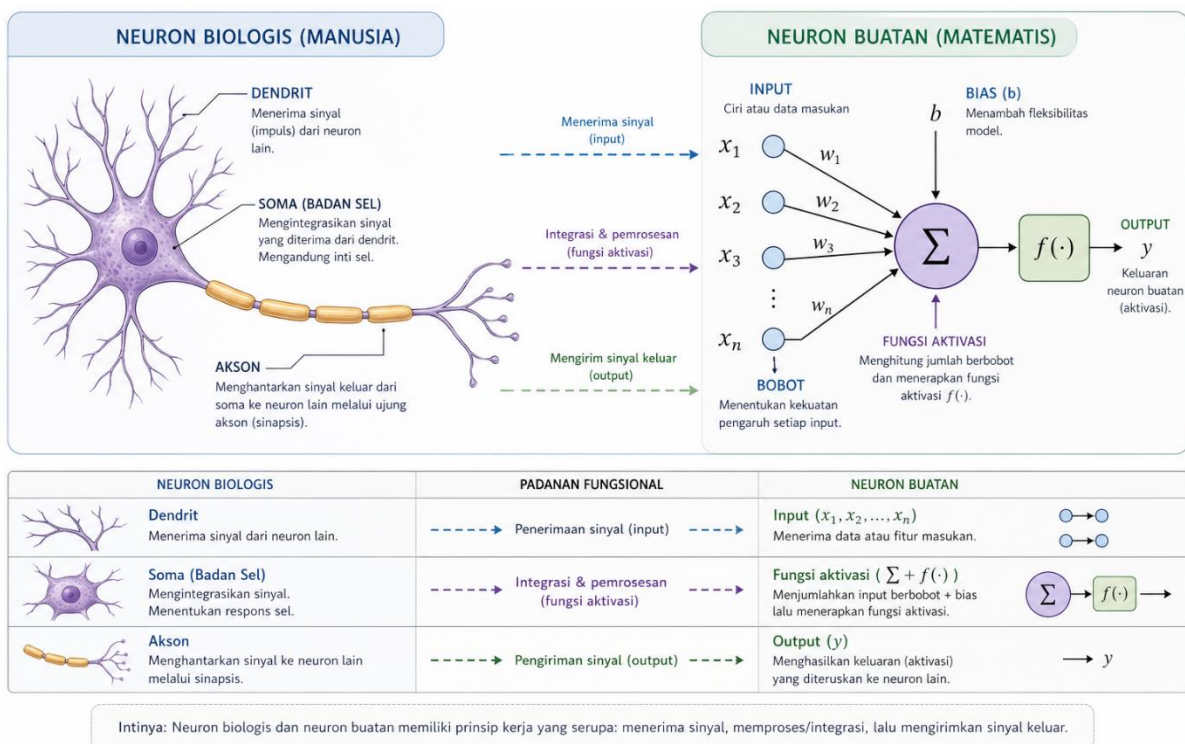


Mata Kuliah

Deep Learning

PENDAHULUAN

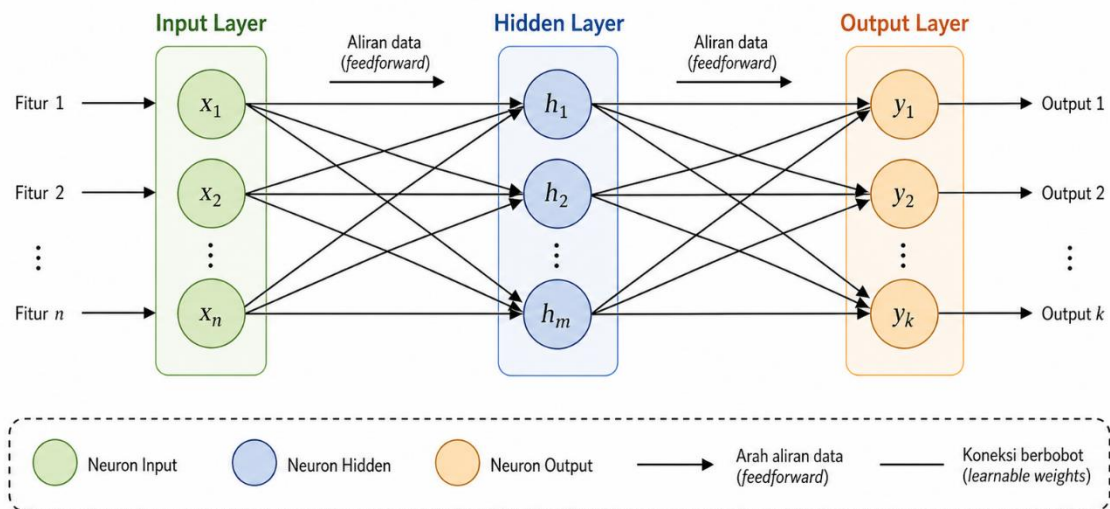
Perkembangan kecerdasan buatan (*Artificial Intelligence*) telah membawa perubahan masif dalam cara komputasi memproses informasi, salah satunya melalui pendekatan *Deep Learning*. Di dalam cabang ilmu tersebut, *Artificial Neural Network* (ANN) atau Jaringan Saraf Tiruan muncul sebagai salah satu model algoritma yang paling adaptif dan kuat. Konsep dasar komputasi ANN sangat terinspirasi oleh cara kerja sistem saraf biologis pada otak manusia, di mana miliaran neuron saling terhubung untuk memproses rangsangan, mempelajari pola, dan mengambil keputusan. Dalam konteks komputasi modern, model ini mereplikasi proses biologis tersebut menggunakan neuron buatan (*node*) yang dihubungkan melalui formulasi matematis.



Gambar 1.1: Perbandingan ilustratif antara struktur anatomi neuron biologis manusia (dendrit, soma, akson) dengan arsitektur matematis pada neuron buatan (input, bobot, fungsi aktivasi).

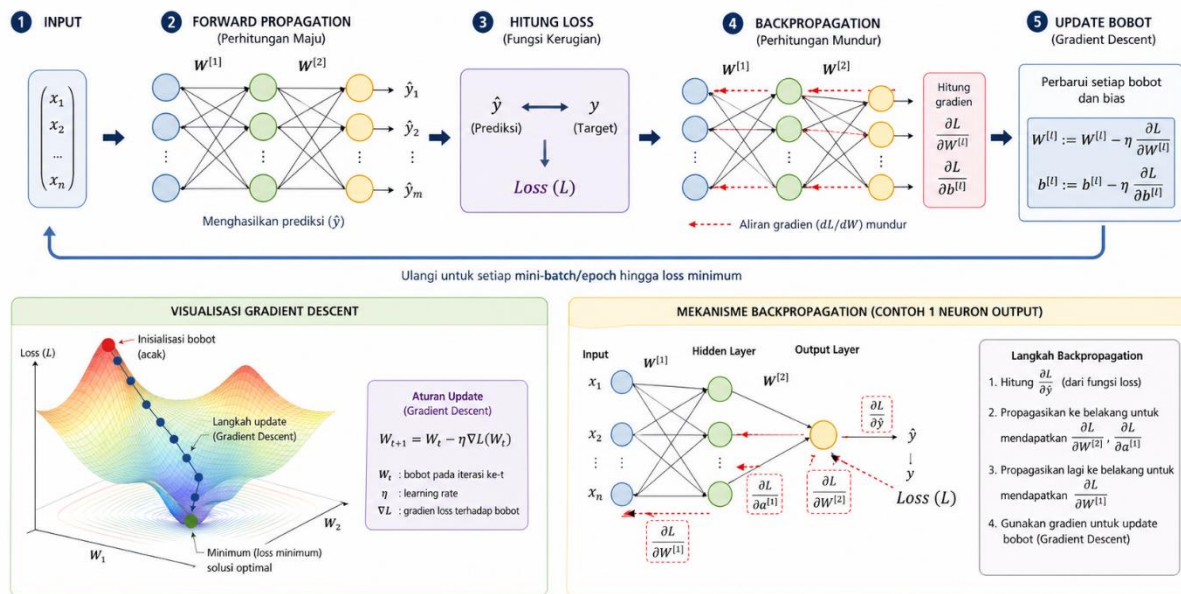
Secara arsitektural, sebuah Jaringan Saraf Tiruan pada umumnya dibangun atas tiga komponen lapisan utama. Komponen pertama adalah *Input Layer* (lapisan masukan) yang berfungsi menerima data mentah dari luar. Selanjutnya terdapat *Hidden Layer* (lapisan tersembunyi)—yang bisa terdiri dari satu atau banyak lapisan—tempat di mana komputasi matematis dan ekstraksi fitur yang kompleks terjadi. Terakhir adalah *Output Layer* (lapisan keluaran) yang bertugas memberikan hasil prediksi, klasifikasi, maupun regresi. Setiap neuron pada satu lapisan terhubung dengan neuron di lapisan berikutnya melalui sebuah jalur koneksi yang memiliki nilai

bobot (*weight*). Nilai bobot inilah yang menentukan seberapa signifikan suatu informasi diproses menuju hasil akhir.



Gambar 1.2: Skema arsitektur dasar Multilayer Perceptron (MLP) yang memvisualisasikan arah aliran data secara feedforward dari Input Layer, melewati Hidden Layer, hingga mencapai Output Layer.

Kehebatan utama dari algoritma ANN terletak pada kemampuannya untuk "belajar" dari kumpulan data latih (*training data*) yang diberikan kepadanya. Proses pembelajaran ini difasilitasi oleh algoritma optimasi, di mana model jaringan akan terus-menerus mengevaluasi nilai kesalahan (*error/loss*) antara prediksi yang ia hasilkan dengan target kebenaran yang sebenarnya. Menggunakan metode yang dikenal luas sebagai *Backpropagation*, jaringan akan melakukan evaluasi dari arah belakang ke depan guna memperbarui nilai bobot dan bias secara iteratif (berulang-ulang). Di samping itu, penyisipan fungsi aktivasi (*activation function*) memungkinkan jaringan ini untuk memahami dan memecahkan permasalahan data non-linear yang sangat rumit, yang sulit diselesaikan oleh algoritma linier tradisional.



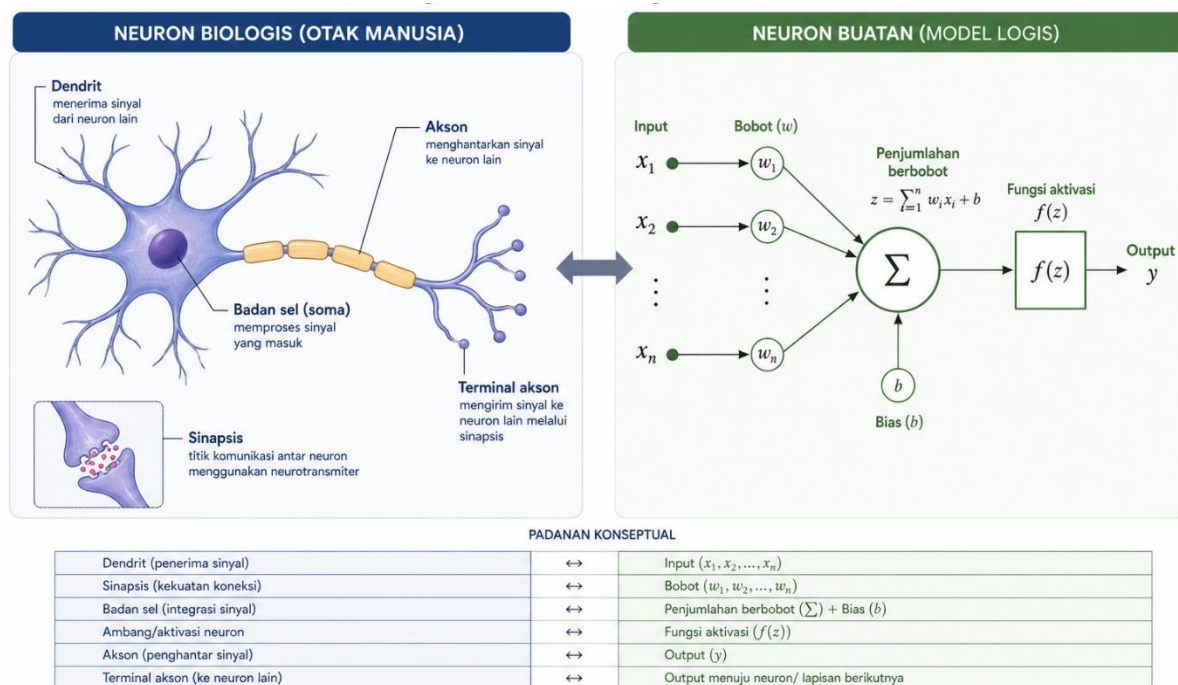
Gambar 1.3: Visualisasi proses optimasi Gradient Descent dan mekanisme Backpropagation dalam memperbarui nilai bobot guna meminimalkan tingkat kesalahan (loss) pada jaringan.

Praktikum *Artificial Neural Network* ini dirancang secara khusus untuk menjembatani kesenjangan antara teori konseptual di ruang kelas dengan implementasi praktis di dunia nyata. Melalui rangkaian modul yang akan dikerjakan, praktikan tidak hanya dituntut untuk memahami persamaan algoritma di balik layar, tetapi juga akan mempraktikkan langsung cara membangun, melatih, serta mengevaluasi model arsitektur jaringan. Dengan memanfaatkan bahasa pemrograman serta *library Machine Learning* berstandar industri (seperti Python, Scikit-Learn, atau TensorFlow/Keras), praktikan diharapkan dapat membangun fondasi analitis yang kuat untuk menyelesaikan berbagai studi kasus nyata berbasis data kompleks di masa depan.

LANDASAN TEORI

2.1 ARTIFICIAL NEURON

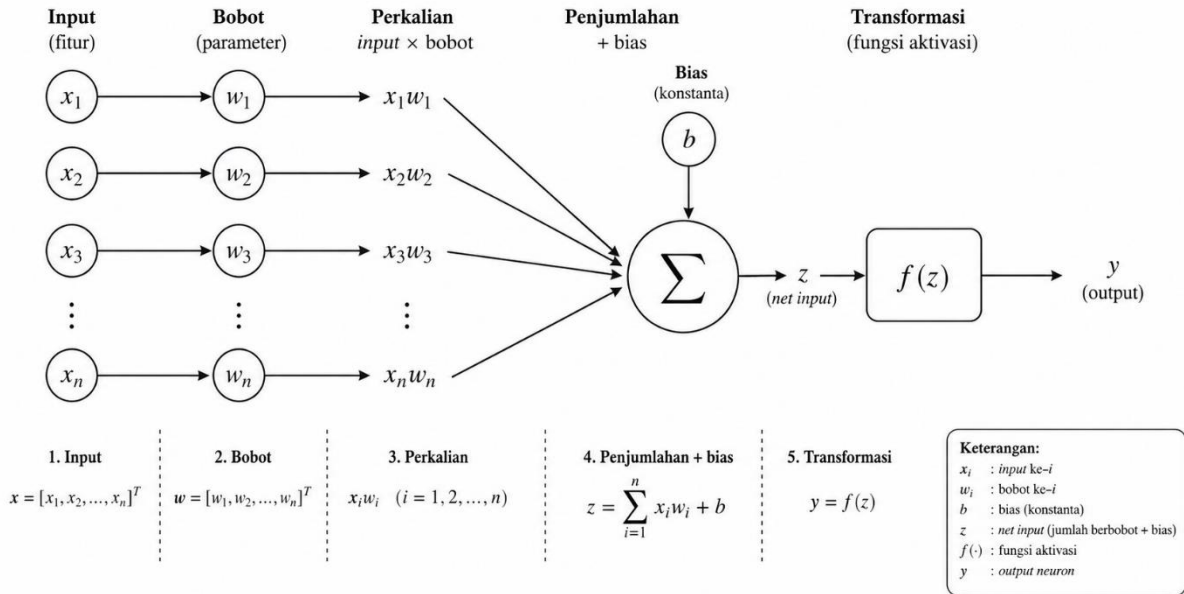
Neuron buatan (*Artificial Neuron*) merupakan pondasi dasar penyusun arsitektur Jaringan Saraf Tiruan. Secara konseptual, unit pemrosesan ini sangat terinspirasi oleh mekanisme kerja neuron biologis pada otak manusia. Dalam sistem biologis, sebuah sel saraf menerima sinyal melalui dendrit, memproses rangsangan tersebut di badan sel, lalu meneruskannya sebagai respons. Serupa dengan hal tersebut, setiap neuron buatan dirancang untuk menerima sekumpulan nilai input dari luar, memprosesnya melalui serangkaian perhitungan matematis di dalam *node*, dan pada akhirnya menghasilkan sebuah nilai output atau prediksi.



Gambar 2.1: Ilustrasi perbandingan antara struktur neuron biologis di otak manusia dengan representasi logis dari neuron buatan.

Secara teknis, proses komputasi di dalam sebuah neuron buatan beroperasi dengan mengombinasikan data masukan dengan parameter-parameter jaringan yang dapat dipelajari (*learnable parameters*). Setiap nilai masukan (x_i) pertama-tama akan dikalikan dengan nilai bobot atau *weight* (w_i) masing-masing. Bobot ini merepresentasikan seberapa penting atau seberapa kuat pengaruh dari suatu input tertentu. Seluruh hasil perkalian tersebut kemudian dijumlahkan dan ditambahkan dengan sebuah parameter penggeser yang disebut *bias* (b), menghasilkan sebuah nilai penjumlahan terboboti (*weighted sum*) yang dilambangkan dengan z . Secara matematis, kalkulasi linier ini dirumuskan sebagai berikut:

$$z = \sum_{i=1}^n w_i \cdot x_i + b = w^T x + b$$



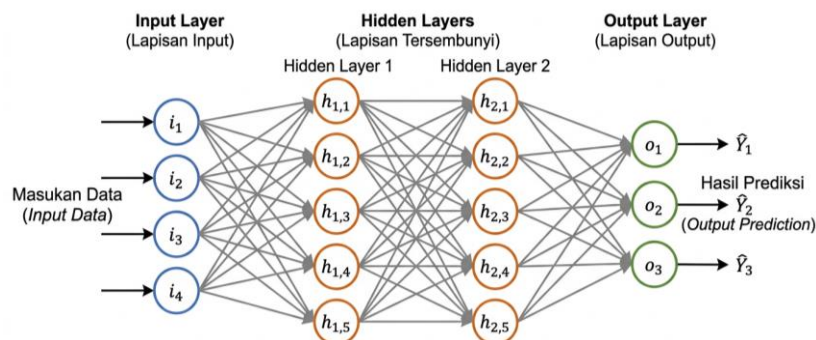
Gambar 2.2: Skema matematis aliran data pada satu neuron buatan yang menampilkan perkalian input dengan bobot, penambahan bias, serta proses transformasi akhir.

Setelah nilai penjumlahan terboboti (z) didapatkan, model tidak langsung menjadikannya sebagai hasil akhir. Nilai z tersebut harus dilewatkan terlebih dahulu pada sebuah fungsi aktivasi (*activation function*) yang dilambangkan dengan $f(z)$. Penambahan fungsi aktivasi ini bertujuan untuk mentransformasikan nilai z ke dalam bentuk keluaran akhir model, sekaligus memberikan batas atau bentuk non-linearitas pada keputusan jaringan. Hasil dari fungsi aktivasi inilah yang menjadi output akhir atau nilai prediksi ($\hat{y}$). Transformasi akhir ini dinyatakan dalam persamaan:

$$\hat{y} = f(z)$$

2.2 ARSITEKTUR ANN (FEEDFORWARD NEURAL NETWORK)

Secara struktural, Jaringan Saraf Tiruan (*Artificial Neural Network*) tidak hanya terdiri dari satu neuron tunggal, melainkan gabungan dari banyak neuron buatan yang disusun dan dihubungkan sedemikian rupa membentuk sebuah jaringan. Salah satu arsitektur dasar dan paling umum digunakan adalah *Feedforward Neural Network* (FNN). Dalam arsitektur ini, informasi atau data mengalir hanya dalam satu arah yang bergerak maju (*feedforward*), dimulai dari titik masukan, melewati lapisan pemrosesan, hingga mencapai titik keluaran. Tidak ada siklus atau aliran balik (*loop*) antar neuron pada arsitektur pendahulu ini, sehingga proses komputasinya berjalan secara berurutan dan terstruktur.



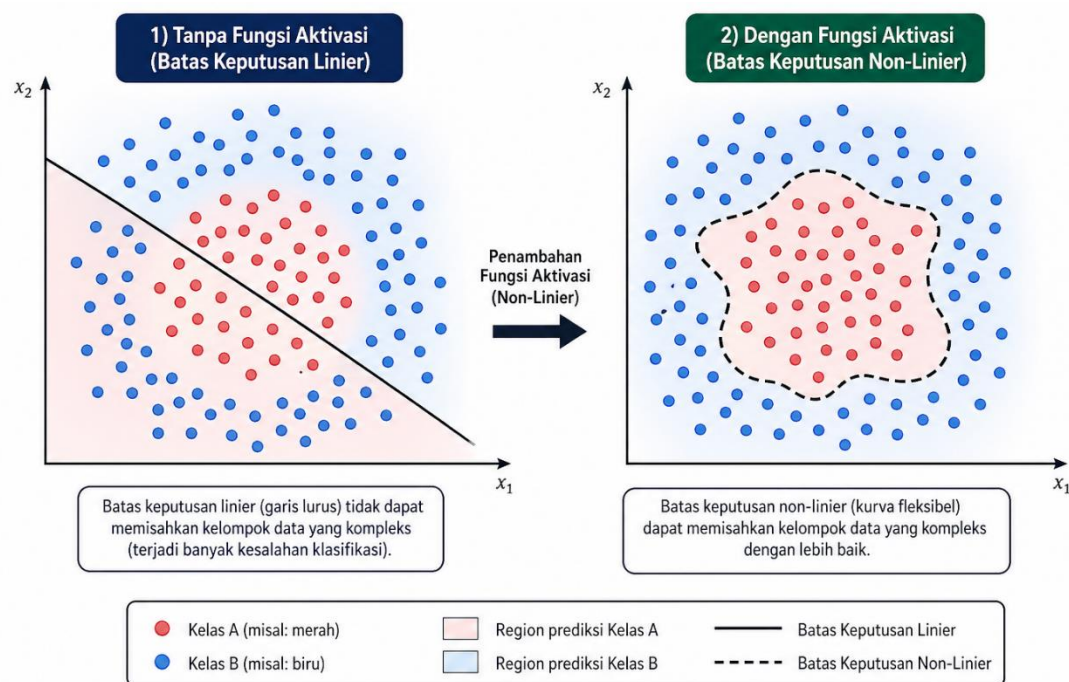
Gambar 2.3: Skema topologi Feedforward Neural Network yang memvisualisasikan aliran data satu arah dari Input Layer, melintasi node pada Hidden Layer, dan berakhir sebagai prediksi pada Output Layer.

Sebuah arsitektur ANN standar tersusun atas tiga kelompok lapisan (*layer*) utama yang saling terhubung secara penuh (*fully connected*). Lapisan pertama adalah *Input Layer* (lapisan masukan) yang bertugas sebagai pintu gerbang masuknya data mentah ke dalam jaringan. Pada lapisan ini, neuron tidak melakukan proses komputasi atau perhitungan matematis apa pun; mereka murni hanya mendistribusikan nilai fitur ke lapisan selanjutnya. Oleh karena itu, jumlah neuron pada *Input Layer* selalu berbanding lurus dan sama persis dengan jumlah fitur (*features*) dari data masukan yang digunakan.

Setelah melewati lapisan masukan, data akan diteruskan ke *Hidden Layer* (lapisan tersembunyi). Dinamakan "tersembunyi" karena lapisan ini tidak berinteraksi langsung dengan lingkungan luar (baik input maupun output target). Di sinilah inti dari proses komputasi terjadi, di mana jaringan melakukan transformasi data dan mengekstraksi representasi pola yang kompleks. Sebuah arsitektur ANN bisa memiliki satu *Hidden Layer* (dikenal sebagai *Shallow Neural Network*) atau lebih dari satu *Hidden Layer* secara berurutan (yang menjadi dasar dari *Deep Learning*). Setelah diproses, hasil akhirnya akan diumpungkan ke *Output Layer* (lapisan keluaran) yang bertugas menghasilkan prediksi atau kesimpulan akhir. Jumlah neuron pada lapisan keluaran ini sangat bergantung pada jenis tugas yang diselesaikan, misalnya satu neuron untuk regresi atau klasifikasi biner, dan beberapa neuron untuk klasifikasi multi-kelas.

2.3 FUNGSI AKTIVASI (ACTIVATION FUNCTION)

Fungsi aktivasi (*Activation Function*) merupakan komponen matematis yang sangat krusial dalam arsitektur Jaringan Saraf Tiruan. Fungsi ini bertugas untuk memproses nilai penjumlahan terbobot (z) dari sebuah neuron dan memutuskan apakah neuron tersebut harus "diaktifkan" atau tidak, serta seberapa kuat sinyal keluaran yang akan diteruskan ke lapisan berikutnya. Peran paling fundamental dari fungsi aktivasi adalah memberikan sifat non-linearitas pada model jaringan. Tanpa adanya fungsi ini, sekompleks apa pun arsitektur *Hidden Layer* yang disusun, sebuah ANN pada akhirnya hanya akan beroperasi sebagai model regresi linier biasa yang tidak mampu mempelajari dan memecahkan pola data yang rumit.

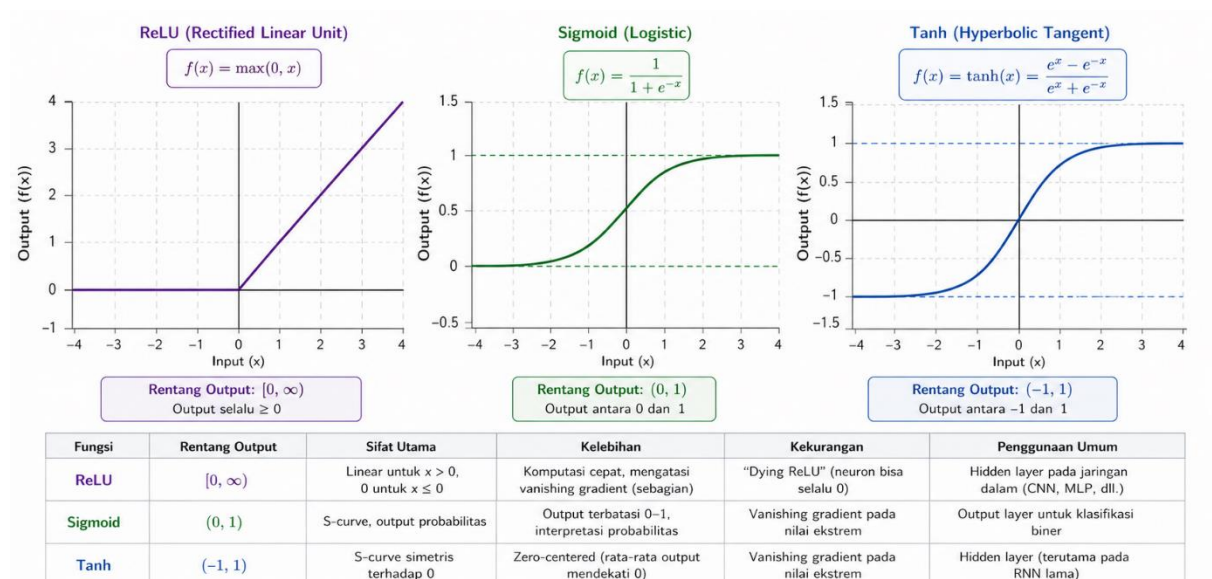


Gambar 2.5: Ilustrasi konseptual bagaimana penambahan fungsi aktivasi mengubah batas keputusan (decision boundary) linier yang kaku menjadi kurva non-linier yang fleksibel untuk memisahkan kelompok data yang kompleks.

Seiring dengan perkembangan ilmu komputasi, telah dikembangkan berbagai jenis fungsi aktivasi yang memiliki karakteristik matematis yang berbeda-beda. Pemilihan fungsi aktivasi yang tepat sangat bergantung pada posisi lapisan di dalam jaringan (apakah berada di *Hidden Layer* atau *Output Layer*) serta jenis permasalahan *Machine Learning* yang sedang diselesaikan. Sebagai contoh, fungsi linier murni tanpa transformasi digunakan untuk memprediksi angka kontinu, sementara fungsi matematis yang menghasilkan nilai berbentuk kurva logistik sangat efektif untuk mengelompokkan probabilitas menjadi sebuah kelas tertentu.

Berikut adalah rangkuman dari beberapa fungsi aktivasi yang paling umum diimplementasikan dalam arsitektur ANN beserta karakteristik penggunaannya:

Fungsi Aktivasi	Formula Matematis	Rentang Output	Kondisi Penggunaan Terbaik
ReLU (Rectified Linear Unit)	$f(z) = \max(0, z)$	$[0, +\infty)$	Sering digunakan pada <i>Hidden layer</i> (paling umum dan efisien).
Sigmoid	$f(z) = \frac{1}{1 + e^{-z}}$	$(0,1)$	Digunakan pada <i>Output layer</i> untuk tugas klasifikasi biner.
Tanh (Hyperbolic Tangent)	$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1,1)$	Digunakan pada <i>Hidden layer</i> sebagai alternatif dari fungsi ReLU.
Softmax	$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0,1)$, total = 1	Digunakan pada <i>Output layer</i> untuk tugas klasifikasi multi-kelas.
Linear	$f(z) = z$	$(-\infty, +\infty)$	Digunakan pada <i>Output layer</i> untuk tugas regresi (prediksi angka kontinu).



Gambar 2.6: Grafik visualisasi dari berbagai fungsi aktivasi (seperti ReLU, Sigmoid, dan Tanh) yang menampilkan perbedaan rentang nilai output pada sumbu-y berdasarkan nilai input pada sumbu-x.

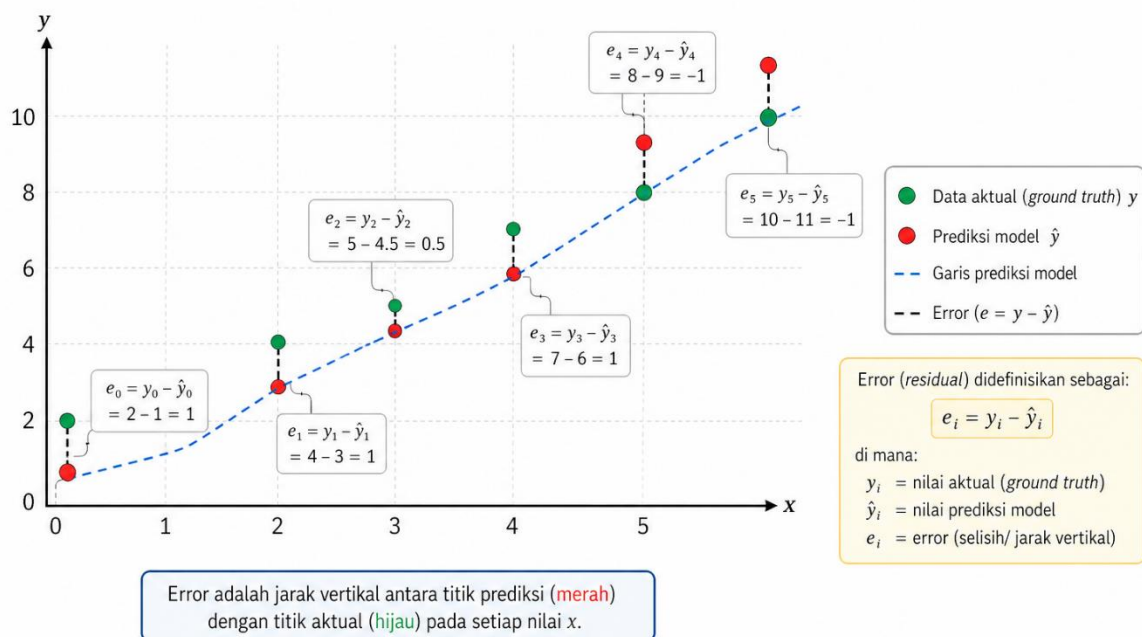
Dalam praktiknya, terdapat beberapa pedoman atau aturan praktis (*rule of thumb*) yang dapat dijadikan acuan utama saat merancang dan melatih Jaringan Saraf Tiruan. Untuk mendesain lapisan tersembunyi (*Hidden Layer*), **ReLU** saat ini menjadi standar pilihan utama yang sangat direkomendasikan karena perhitungannya yang cepat dan kemampuannya dalam meminimalkan masalah gradien yang menghilang (*vanishing gradient*).

Sementara itu, desain lapisan keluaran (*Output Layer*) selalu disesuaikan secara mutlak dengan target hasil akhir:

- Jika model ditujukan untuk **regresi**, gunakan **fungsi Linear** (tanpa aktivasi tambahan).
- Jika model ditujukan untuk **klasifikasi biner** (memilih antara dua hal), gunakan fungsi **Sigmoid**.
- Jika model ditujukan untuk **klasifikasi multi-kelas** (memilih lebih dari dua kategori), terapkan fungsi **Softmax** (atau tanpa aktivasi jika fungsi *loss* yang digunakan sudah merangkap penghitungan probabilitas seperti *CrossEntropyLoss*).

2.4 FUNGSI KERUGIAN (LOSS FUNCTION)

Dalam proses pelatihan Jaringan Saraf Tiruan, *Loss Function* atau fungsi kerugian memegang peranan krusial sebagai indikator utama untuk mengukur kinerja model. Fungsi ini bertugas mengevaluasi secara kuantitatif seberapa jauh nilai prediksi yang dihasilkan oleh jaringan melenceng dari nilai target aktual yang sebenarnya. Pemahaman dasarnya sangat sederhana: semakin kecil nilai *loss* yang dihitung, semakin akurat model tersebut dalam memetakan input menjadi output yang benar. Sebaliknya, nilai *loss* yang tinggi menjadi sinyal bahwa jaringan masih belum "paham" dan perlu memperbarui parameter bobotnya secara signifikan.

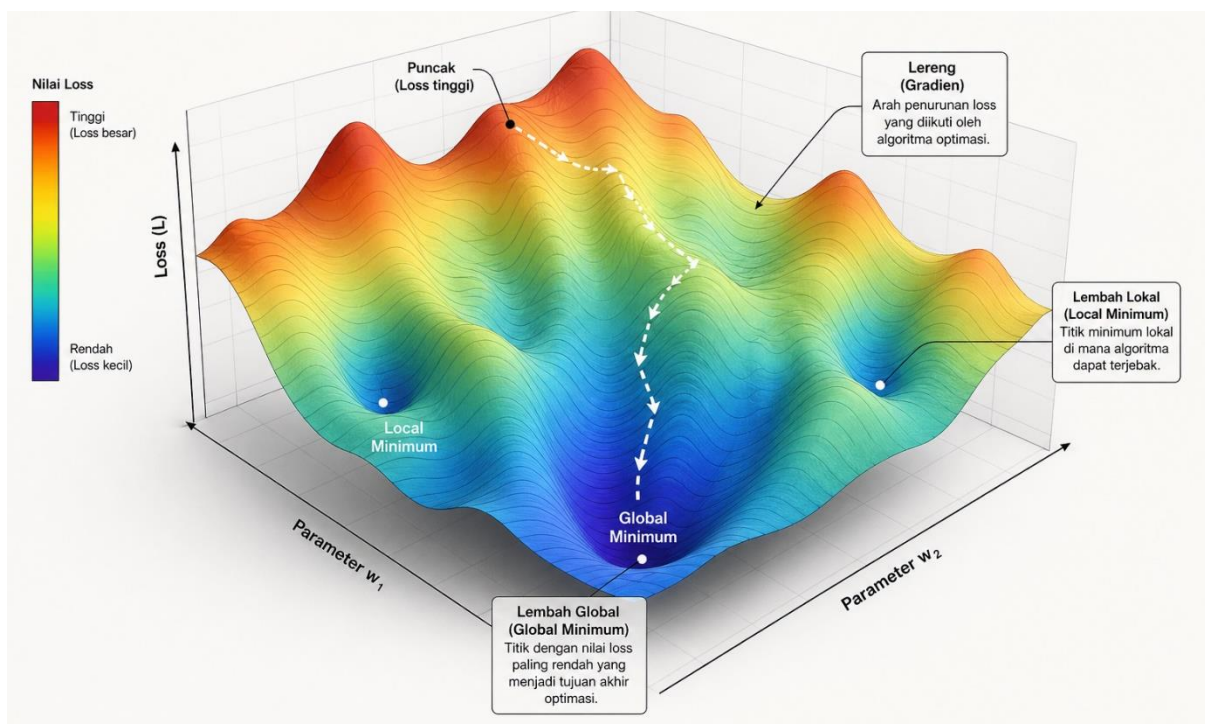


Gambar 2.7: Ilustrasi perhitungan error yang memvisualisasikan jarak atau selisih antara titik prediksi model dengan titik data kebenaran aktual (*ground truth*) pada sebuah grafik.

Pemilihan jenis *Loss Function* tidak bersifat universal; ia harus disesuaikan secara spesifik dengan jenis permasalahan *Machine Learning* yang sedang diselesaikan. Mengukur kesalahan pada prediksi angka kontinu (regresi) membutuhkan pendekatan matematis yang sangat berbeda dibandingkan dengan membandingkan probabilitas kategori (klasifikasi). Di dalam ekosistem pengembangan modern berbasis Python, seperti kerangka kerja PyTorch, fungsi-fungsi kerugian ini telah distandardisasi dan dapat diakses dengan mudah untuk mempercepat proses pembuatan model.

Berikut adalah tabel pemetaan berbagai *Loss Function* berdasarkan jenis masalahnya beserta representasi modulnya di dalam PyTorch:

Jenis Masalah	Loss Function	Modul PyTorch
Regresi	<i>Mean Squared Error</i> (MSE)	nn.MSELoss()
Regresi	<i>Mean Absolute Error</i> (MAE)	nn.L1Loss()
Klasifikasi Biner	<i>Binary Cross-Entropy</i>	nn.BCELoss() atau nn.BCEWithLogitsLoss()
Klasifikasi Multi-Kelas	<i>Cross-Entropy</i>	nn.CrossEntropyLoss()



Gambar 2.8: Visualisasi topografi permukaan fungsi kerugian (*loss landscape*), memperlihatkan lereng dan lembah di mana algoritma optimasi berusaha mencari titik nilai kerendah (*global minimum*).

Secara analitis, fungsi-fungsi tersebut beroperasi melalui persamaan matematis yang dirancang untuk memberikan penalti yang tepat terhadap kesalahan. Pada permasalahan regresi, metode evaluasi yang paling banyak diadopsi adalah *Mean Squared Error* (MSE). Metode ini bekerja dengan menghitung selisih antara nilai aktual (y_i) dengan nilai prediksi ($\hat{y}_i$), lalu mengkuadratkan selisih tersebut guna memberikan penalti yang jauh lebih besar untuk kesalahan yang ekstrem, dan menghitung rata-ratanya untuk seluruh data (n):

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Sementara itu, untuk permasalahan klasifikasi biner yang keluarannya berbentuk probabilitas antara rentang 0 hingga 1, pengukuran kesalahan jarak linier tidak lagi relevan. Pendekatan yang digunakan adalah *Binary Cross-Entropy*, yang menerapkan fungsi logaritma. Fungsi ini mengevaluasi seberapa yakin model memprediksi probabilitas untuk dua kelas yang saling bertolak belakang:

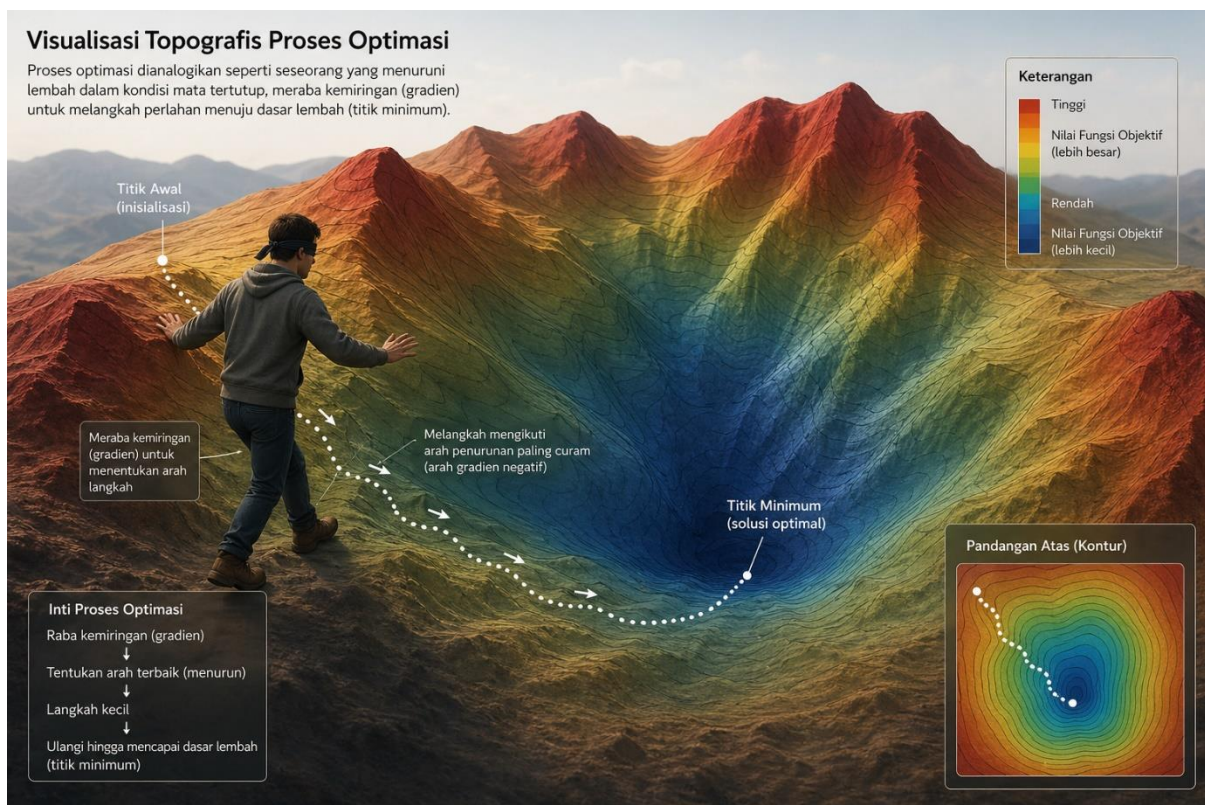
$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Apabila klasifikasi yang dilakukan melibatkan pemisahan ke dalam lebih dari dua kelas yang berdiri sendiri (*multi-class classification*), maka perhitungan fungsi *loss* diperluas menjadi *Cross-Entropy* umum. Evaluasi ini menjumlahkan logaritma dari probabilitas yang diprediksi hanya pada kelas yang benar, di mana C merepresentasikan total seluruh kategori kelas yang ada:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

2.5 OPTIMIZER

Dalam arsitektur Jaringan Saraf Tiruan, setelah model mengevaluasi tingkat kesalahannya melalui *Loss Function*, langkah krusial berikutnya adalah bertindak untuk memperbaiki kesalahan tersebut. Di sinilah *Optimizer* atau algoritma pengoptimasi mengambil peran utama. Secara teknis, *optimizer* bertugas untuk memperbarui parameter-parameter jaringan yang dapat dipelajari, khususnya nilai bobot (*weight*) dan bias, berdasarkan nilai gradien yang dihitung selama proses propagasi balik (*backpropagation*). Tujuan akhir dari mekanisme ini adalah menuntun model bergerak selangkah demi selangkah mencari nilai kesalahan yang paling minimum.

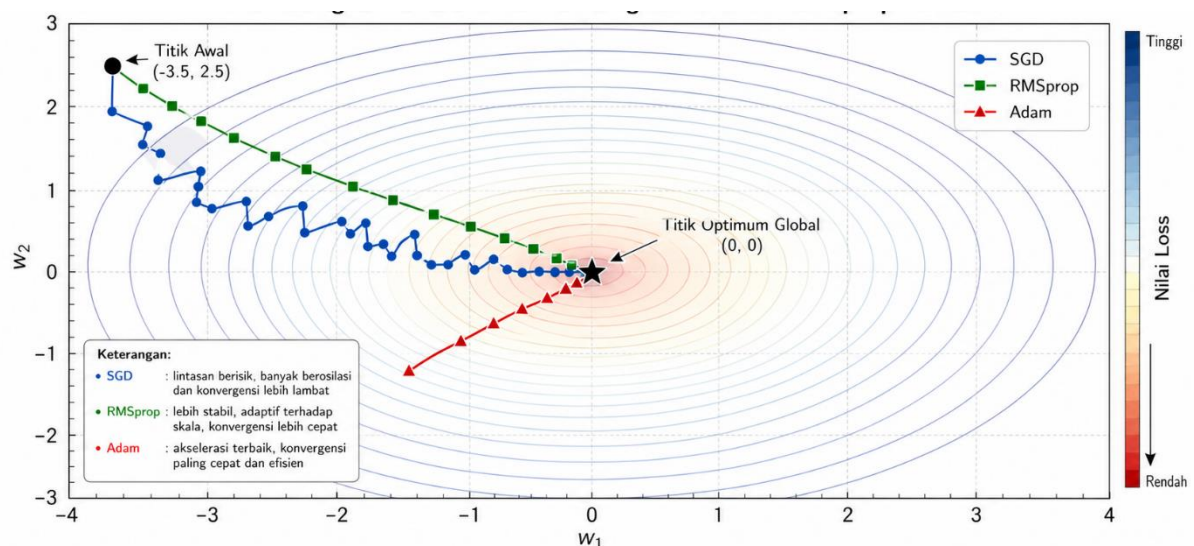


Gambar 2.9: Visualisasi topografis proses optimasi yang sering dianalogikan seperti seseorang yang menuruni lembah dalam kondisi mata tertutup, meraba kemiringan (gradien) untuk melangkah perlahan menuju dasar lembah (titik minimum).

Seiring dengan pesatnya riset di bidang *Deep Learning*, telah dikembangkan berbagai jenis algoritma *optimizer* dengan karakteristik matematis dan heuristik yang bervariasi. Beberapa algoritma dirancang untuk beroperasi secara sederhana dan stabil, sementara yang lain disematkan fungsi momentum komputasi adaptif guna mempercepat laju konvergensi (kecepatan mencapai titik optimum) dan mencegah model terjebak pada *local minima* (titik minimum palsu). Di dalam ekosistem pemrograman modern seperti PyTorch, beragam fungsi pengoptimasi ini telah diformulasikan menjadi modul praktis yang siap diimplementasikan.

Berikut adalah tabel yang membandingkan beberapa *Optimizer* utama yang sering diandalkan oleh para praktisi, beserta modul panggilannya di PyTorch:

Optimizer	Keterangan dan Karakteristik Utama	Modul PyTorch
SGD	<i>Stochastic Gradient Descent</i> — Algoritma dasar yang beroperasi secara sederhana, stabil, dan dinilai sangat efektif untuk model standar.	<code>torch.optim.SGD()</code>
Adam	<i>Adaptive Moment Estimation</i> — Algoritma paling populer saat ini yang mengombinasikan momentum, berkinerja cepat konvergen, dan sering menjadi standar (<i>default</i>) pada banyak proyek.	<code>torch.optim.Adam()</code>
RMSprop	<i>Root Mean Square Propagation</i> — Algoritma yang sangat adaptif dan memiliki rekam jejak yang sangat baik, terutama untuk menganalisis data yang berurutan atau non-stasioner.	<code>torch.optim.RMSprop()</code>



Gambar 2.10: Grafik perbandingan lintasan arah konvergensi antara algoritma SGD, RMSprop, dan Adam saat bergerak dari titik awal menuju titik optimum global, memperlihatkan perbedaan akselerasi masing-masing pengoptimasi.

Fondasi mekanika pergerakan yang mendasari sebagian besar algoritma *optimizer* ini berakar pada prinsip dasar *Gradient Descent*. Logika matematisnya berpusat pada pengurangan atau penggeseran nilai bobot lama (w_{lama}) berlawanan dengan arah nilai turunan dari fungsi kerugian terhadap bobot tersebut ($\frac{\partial \mathcal{L}}{\partial w}$). Besarnya langkah koreksi bobot pada setiap iterasi tidak dibiarkan liar, melainkan dikendalikan secara ketat oleh sebuah parameter penyesuai (*hyperparameter*) utama yang disebut sebagai *Learning Rate*, dilambangkan dengan huruf Yunani η (eta).

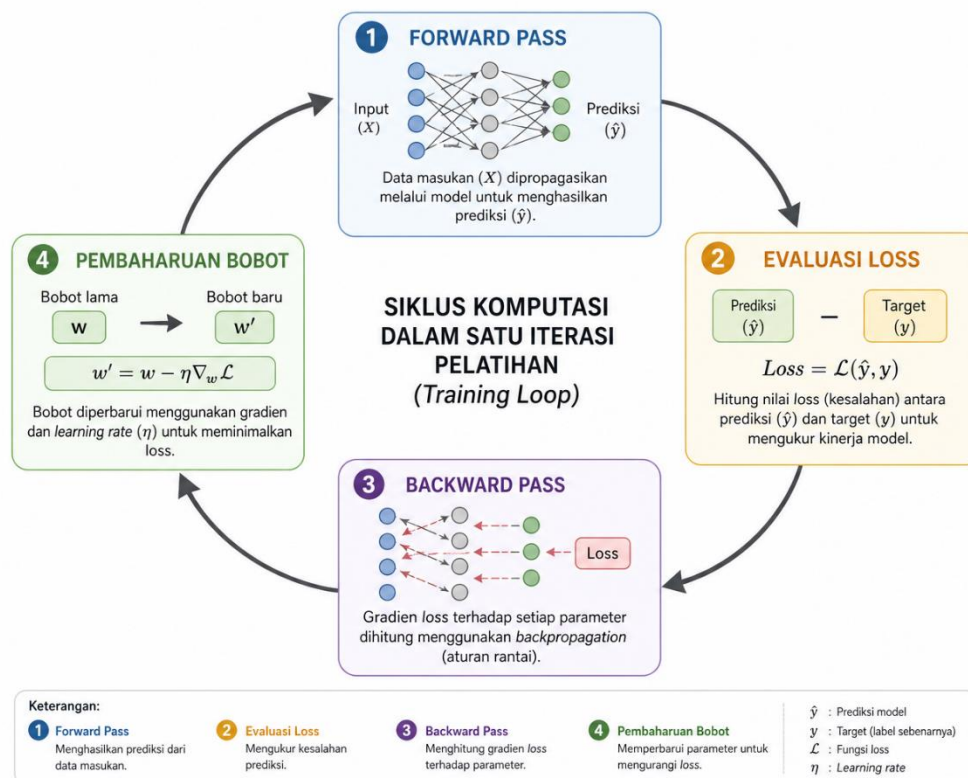
Secara kalkulatif, persamaan pembaruan bobot tersebut dirumuskan pada persamaan berikut:

$$w_{baru} = w_{lama} - \eta \cdot \frac{\partial \mathcal{L}}{\partial w}$$

Dalam konstelasi pelatihan ANN, penentuan besaran *Learning Rate* (η) merupakan seni komputasi tersendiri karena ia sangat mendikte seberapa besar atau kecil langkah model untuk "belajar". Apabila parameter η diatur terlalu besar, lompatan pembaharuan bobot akan terlalu drastis sehingga model cenderung melompati lembah optimum (*overshooting*) dan gagal mencapai konvergensi. Sebaliknya, apabila ia dipatok terlalu kecil, laju pembaruan akan sangat lambat, membuat proses pelatihan memakan waktu yang sangat panjang atau bahkan terhenti sebelum waktunya.

2.6 PROSES PELATIHAN ANN

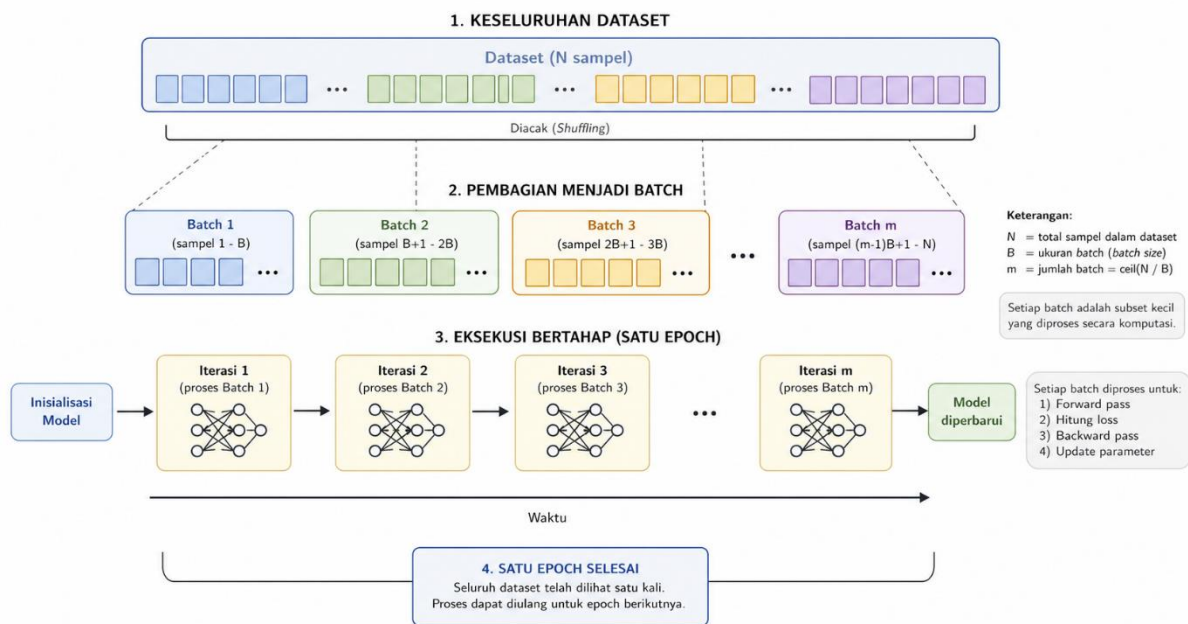
Pelatihan Jaringan Saraf Tiruan pada dasarnya adalah sebuah siklus pembelajaran iteratif di mana model secara bertahap menyesuaikan parameter internalnya untuk meminimalkan kesalahan prediksi. Proses ini mengintegrasikan seluruh komponen matematis yang telah dibahas sebelumnya—mulai dari arsitektur jaringan, fungsi aktivasi, kalkulasi *loss function*, hingga langkah optimasi—ke dalam satu alur komputasi yang berkesinambungan. Alur komputasi ini dirancang sedemikian rupa agar jaringan secara otomatis dapat mengekstrak pola dari data latih dan memetakan hubungan yang kompleks antara fitur masukan dan target keluaran yang diharapkan.



Gambar 2.11: Siklus komputasi dalam satu iterasi pelatihan (*Training Loop*) yang menunjukkan aliran proses dari *Forward Pass*, evaluasi *Loss*, *Backward Pass*, hingga tahap akhir pembaruan bobot.

Secara prosedural, siklus pelatihan (yang sering disebut sebagai *Training Loop*) untuk satu kelompok data terdiri dari empat tahapan utama yang berjalan secara berurutan. Pertama adalah tahapan *Forward Pass*, di mana data masukan mengalir maju menembus lapisan-lapisan jaringan hingga menghasilkan prediksi awal ($\hat{y} = f(x)$). Kedua, nilai prediksi tersebut dievaluasi menggunakan *Loss Function* untuk menghitung seberapa besar tingkat kesalahan ($\mathcal{L}$) dibandingkan dengan label data aktual. Ketiga, model mengeksekusi *Backward Pass* (propagasi balik) guna menghitung nilai turunan atau gradien dari kesalahan tersebut terhadap setiap bobot parameter ($\frac{\partial \mathcal{L}}{\partial w}$). Keempat dan terakhir, *Optimizer* bertugas untuk memperbarui

bobot-bobot jaringan ($w = w - \eta \nabla \mathcal{L}$) guna memangkas nilai kesalahan pada prediksi di iterasi selanjutnya.



Gambar 2.12: Ilustrasi hierarki pembagian keseluruhan dataset menjadi beberapa subset komputasi yang lebih kecil (batch) yang dieksekusi secara bertahap untuk menyelesaikan satu siklus pembelajaran utuh (epoch).

Mengingat ukuran dataset komprehensif di dunia nyata sering kali terlalu besar untuk diproses seluruhnya secara bersamaan oleh memori perangkat keras, rangkaian empat tahapan komputasi di atas umumnya dipecah dan dieksekusi ke dalam subset-subset data yang lebih kecil. Proses pembaruan bobot ini akan diulang terus-menerus hingga seluruh data latih habis diproses, dan siklus besar tersebut akan diulangi lagi hingga ribuan kali demi memastikan model mencapai tingkat konvergensi dan akurasi maksimal. Untuk mengontrol dan memahami dinamika skala pelatihan jaringan ini, terdapat beberapa terminologi penting yang menjadi standar operasional pemodelan:

- **Epoch:** Merupakan sebuah siklus penuh di mana seluruh dataset latih tanpa terkecuali telah dilewatkan secara utuh (baik maju maupun mundur) melalui jaringan sebanyak satu kali.
- **Batch:** Merupakan subset atau bagian kecil dari keseluruhan data latih yang diproses sekaligus secara bersama-sama dalam satu kali eksekusi komputasi.
- **Iteration (Iterasi):** Merupakan satu kali proses pembaruan parameter bobot oleh *optimizer* setelah memproses satu *batch* data. Sebagai rumus praktis, 1 *epoch* sama dengan jumlah total data dibagi dengan *batch size*.
- **Learning Rate:** Merupakan parameter penentu (*hyperparameter*) tingkat kecepatan belajar model, yang mengatur seberapa agresif atau konservatif langkah perubahan bobot yang diterapkan pada setiap iterasi.

PRAKTIKUM

3.1 DEPENDENSI

CODE

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_moons, make_classification
from sklearn.preprocessing import StandardScaler

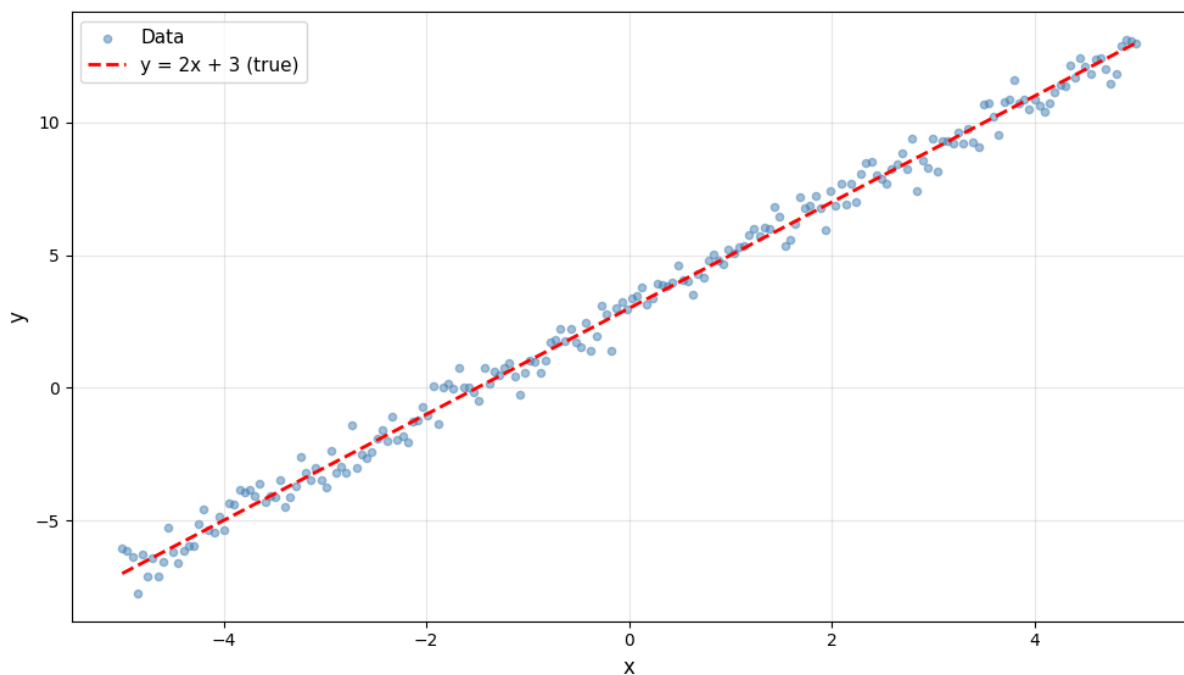
# Cek versi PyTorch
print(f"PyTorch Version: {torch.__version__}")

# Cek ketersediaan GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {device}")
```


3.2 REGRESI LINIER SEDERHANA (SIMPLE LINEAR REGRESSION)

3.2.A. MEMBUAT DATASET

Bagian pertama dari kode bertujuan untuk membuat dataset sintesis yang akan digunakan dalam pemodelan regresi linear. Fungsi `torch.manual_seed(42)` digunakan untuk memastikan bahwa nilai acak yang dihasilkan akan selalu sama setiap kali kode dijalankan. Sebanyak 200 titik data dibuat menggunakan `torch.linspace` untuk variabel independen x dengan rentang antara -5 hingga 5. Variabel dependen y dihasilkan berdasarkan persamaan linear $y = 2x + 3$, yang kemudian ditambahkan sedikit *noise* acak dari distribusi normal menggunakan `torch.randn`. Setelah data berhasil dibuat, pustaka `matplotlib.pyplot` digunakan untuk memvisualisasikan persebaran data tersebut dalam bentuk diagram pencar (*scatter plot*), beserta garis putus-putus merah yang merepresentasikan persamaan fungsi asli tanpa penambahan *noise*.



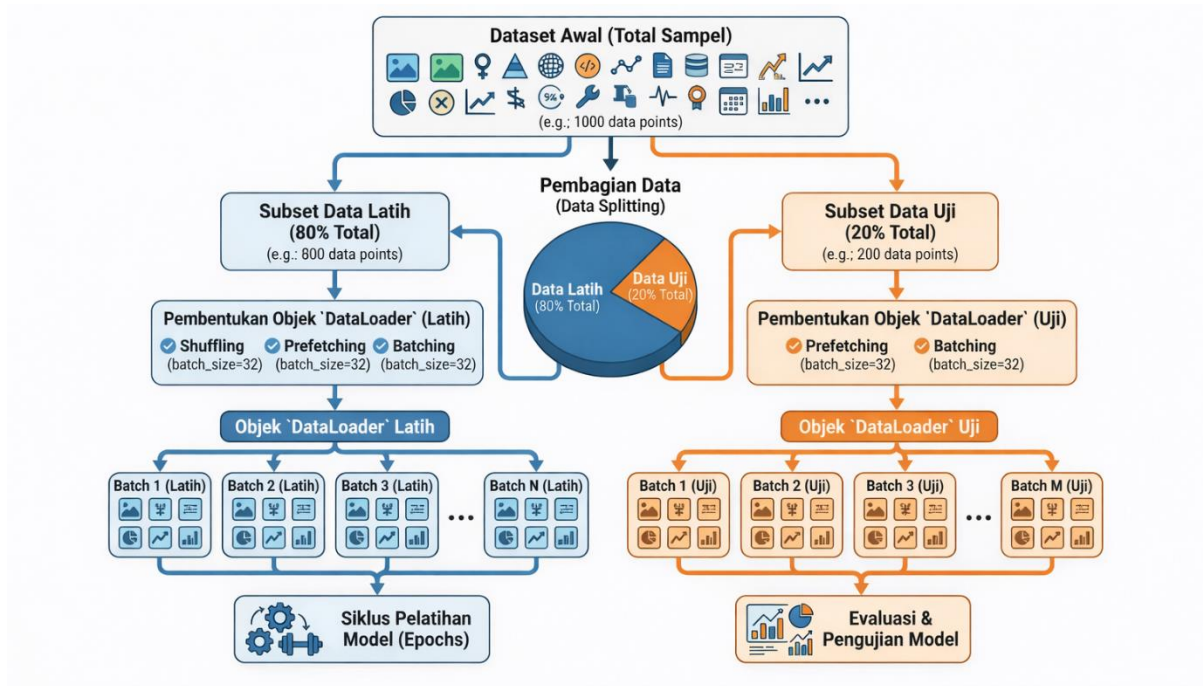
Gambar 3.1: Visualisasi dataset regresi linear sintetis dengan persamaan $y = 2x + 3$ yang disertai penambahan *noise* Gaussian.

CODE

```
# =====  
# DATASET:  $y = 2x + 3 + \text{noise}$   
# =====  
  
torch.manual_seed(42) # Agar hasil reproducible  
  
# Membuat 200 data point  
n_samples = 200  
X = torch.linspace(-5, 5, n_samples).reshape(-1, 1) # Shape: (200, 1)  
y_true = 2 * X + 3 + torch.randn(n_samples, 1) * 0.5 # Tambah noise  
  
# Visualisasi data  
plt.figure(figsize=(10, 6))  
plt.scatter(X.numpy(), y_true.numpy(), alpha=0.5, s=20,  
            color='steelblue', label='Data')  
plt.plot(X.numpy(), (2*X + 3).numpy(), 'r--', linewidth=2, label='y = 2x  
+ 3 (true)')  
plt.xlabel('x', fontsize=12)  
plt.ylabel('y', fontsize=12)  
plt.title('Dataset Regresi Linear', fontsize=14, fontweight='bold')  
plt.legend(fontsize=11)  
plt.grid(True, alpha=0.3)  
plt.tight_layout()  
plt.show()
```

3.2.B. PEMBAGIAN DATA & BUAT DATALOADER

Kode pada sel kedua berfokus pada pembagian dataset menjadi dua bagian, yaitu data latih (*training data*) dan data uji (*testing data*). Pembagian dilakukan dengan rasio 80% untuk pelatihan dan 20% untuk pengujian. Indeks data diacak terlebih dahulu menggunakan `torch.randperm` untuk memastikan tidak ada bias urutan data selama proses pelatihan. Data kemudian dipisahkan sesuai dengan indeks acak tersebut. Setelah itu, data latih dibungkus menggunakan `TensorDataset` dan `DataLoader`. Penggunaan `DataLoader` ini sangat penting karena memungkinkan pemrosesan data secara berangsur-angsur dalam ukuran *batch* (dalam hal ini berukuran 32) dan mengacak ulang urutan data secara otomatis setiap kali satu putaran pelatihan (*epoch*) selesai.



Gambar 3.2: Proses pembagian (splitting) dataset menjadi data latih (80%) dan data uji (20%) serta pembentukan objek DataLoader untuk pemrosesan batching.

CODE

```
# =====
# SPLIT DATA: 80% Train, 20% Test
# =====

# Shuffle indices
indices = torch.randperm(n_samples)
train_size = int(0.8 * n_samples)

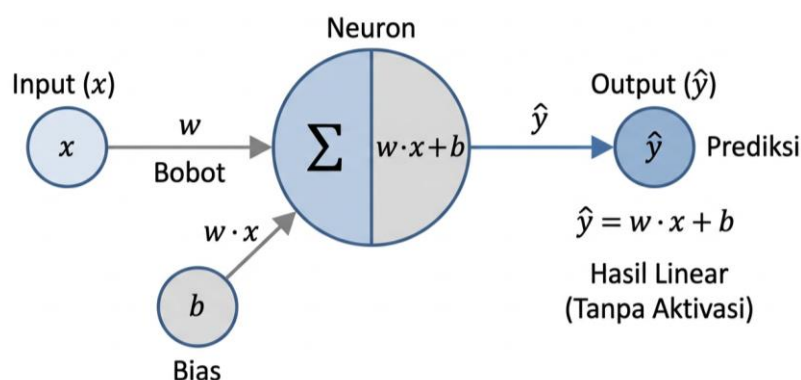
X_train = X[indices[:train_size]]
y_train = y_true[indices[:train_size]]
X_test = X[indices[train_size:]]
y_test = y_true[indices[train_size:]]

print(f"Data Training: {X_train.shape[0]} sampel")
print(f"Data Testing : {X_test.shape[0]} sampel")

# Membuat DataLoader
train_dataset = TensorDataset(X_train, y_train)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
```

3.2.C. MEMBANGUN MODEL

Selanjutnya adalah tahap pendefinisian arsitektur model regresi linear menggunakan PyTorch. Model dibangun dengan metode `nn.Sequential` yang di dalamnya memuat sebuah lapisan `nn.Linear(in_features=1, out_features=1)`. Lapisan ini merepresentasikan satu neuron yang menerima satu nilai masukan dan menghasilkan satu nilai keluaran. Secara matematis, model ini bertugas mempelajari fungsi prediksi $\hat{y} = wx + b$, di mana w melambangkan bobot (*weight*) dan b melambangkan bias. Fungsi `print` kemudian digunakan untuk menampilkan arsitektur model, jumlah parameter yang perlu dilatih (terdapat dua parameter: satu bobot dan satu bias), serta nilai inisialisasi awal dari parameter-parameter tersebut sebelum proses pelatihan dimulai.



Gambar 3.3: Arsitektur jaringan saraf satu neuron (tanpa fungsi aktivasi) yang merepresentasikan persamaan linear $\hat{y} = wx + b$.

CODE

```
# =====  
# MODEL: Regresi Linear (1 neuron)  
# =====  
  
# Arsitektur:  
#   Input (1 fitur) → Linear(1, 1) → Output (1 nilai)  
#  
# Ini setara dengan:  $\hat{y} = w \cdot x + b$  (1 neuron tanpa aktivasi)  
  
model_regresi = nn.Sequential(  
    nn.Linear(in_features=1, out_features=1)  # 1 input → 1 output  
)  
  
print("Arsitektur Model:")  
print(model_regresi)  
print(f"\nJumlah parameter: {sum(p.numel() for p in  
model_regresi.parameters())}")  
# Output: 2 parameter (1 weight + 1 bias)  
  
# Lihat parameter awal (sebelum training)  
for name, param in model_regresi.named_parameters():  
    print(f"  {name}: {param.data}")
```

3.2.D. MENENTUKAN LOSS FUNCTION & OPTIMIZER

Pada bagian ini, komponen evaluasi kesalahan dan algoritma pembaruan bobot ditentukan. *Loss function* yang digunakan adalah *Mean Squared Error* (MSE), yang diinisialisasi melalui perintah `nn.MSELoss()`. Fungsi ini akan mengukur seberapa jauh perbedaan atau kuadrat sisa antara nilai prediksi yang dihasilkan oleh model dengan nilai target sebenarnya. Selain itu, fungsi optimisasi *Stochastic Gradient Descent* (SGD) didefinisikan menggunakan `optim.SGD` dengan *learning rate* sebesar 0.01. Optimizer tersebut bertugas untuk memperbarui nilai parameter model pada setiap iterasi perhitungan sehingga tingkat kesalahan (*loss*) menjadi semakin minimal.

CODE

```
# =====  
# LOSS FUNCTION & OPTIMIZER  
# =====  
  
# MSE Loss untuk regresi  
criterion = nn.MSELoss()  
  
# Optimizer: SGD dengan learning rate 0.01  
optimizer = optim.SGD(model_regresi.parameters(), lr=0.01)
```

3.2.E. PROSES PELATIHAN (TRAINING LOOP)

Proses pelatihan model atau *training loop* dijalankan pada blok kode ini dengan durasi sebanyak 100 *epoch*. Pada setiap *epoch*, data latih diproses per *batch* melalui serangkaian tahapan utama. Pertama, model melakukan *forward pass* untuk memprediksi nilai keluaran berdasarkan data masukan. Kedua, nilai *loss* dihitung dengan membandingkan hasil prediksi terhadap data target nyata. Ketiga, metode `optimizer.zero_grad()` dipanggil untuk mengosongkan nilai gradien dari iterasi sebelumnya agar tidak berakumulasi. Selanjutnya, metode `loss.backward()` menjalankan komputasi *backward pass* untuk menghitung turunan (gradien) kesalahan terhadap masing-masing parameter. Terakhir, `optimizer.step()` menyesuaikan nilai parameter berdasarkan gradien yang telah diperoleh. Rata-rata nilai *loss* dari setiap *epoch* kemudian disimpan ke dalam sebuah daftar bernama `train_losses` untuk keperluan pemantauan, dan program akan mencetak nilai *loss* ke layar setiap kelipatan 10 *epoch*.

CODE

```
# =====
# TRAINING
# =====

num_epochs = 100
train_losses = []

print("Mulai Training...")
print("-" * 50)

for epoch in range(num_epochs):
    epoch_loss = 0.0
    num_batches = 0

    for X_batch, y_batch in train_loader:
        # 1. Forward pass: hitung prediksi
        y_pred = model_regresi(X_batch)

        # 2. Hitung loss
        loss = criterion(y_pred, y_batch)

        # 3. Backward pass: hitung gradien
        optimizer.zero_grad() # Reset gradien ke nol
        loss.backward()       # Hitung gradien

        # 4. Update bobot
        optimizer.step()

    epoch_loss += loss.item()
    num_batches += 1

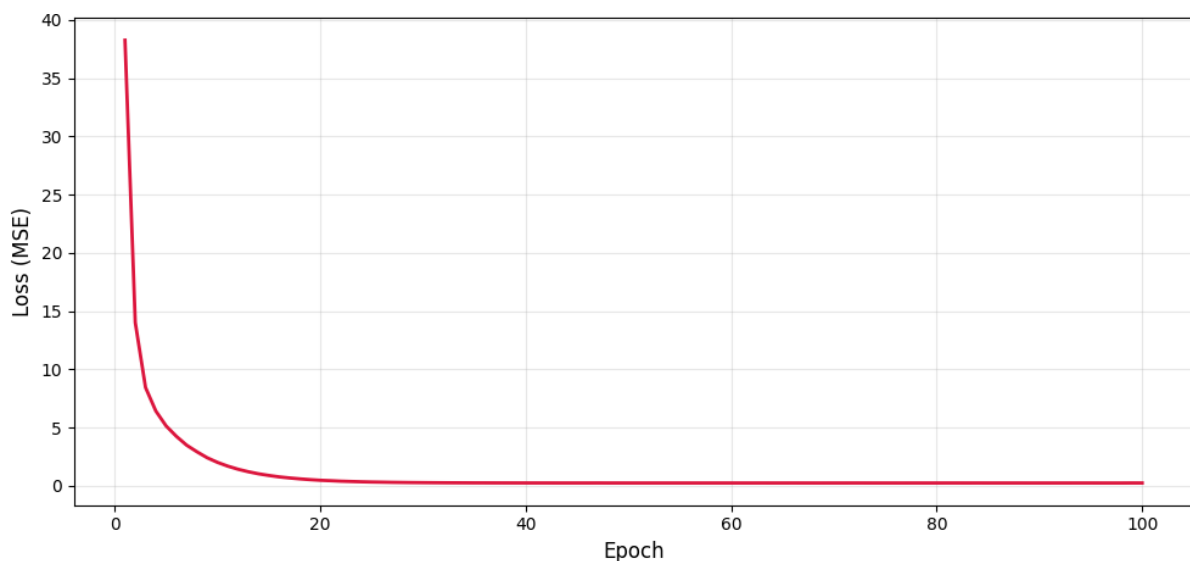
    avg_loss = epoch_loss / num_batches
    train_losses.append(avg_loss)

    # Print setiap 10 epoch
    if (epoch + 1) % 10 == 0:
        print(f"Epoch [{epoch+1:3d}/{num_epochs}] | Loss:
{avg_loss:.4f}")

print("-" * 50)
print("Training Selesai!")
```

3.2.F. VISUALISASI KURVA PELATIHAN

Blok kode keenam ini dirancang untuk memvisualisasikan kurva penurunan kesalahan (*training loss*) selama proses pelatihan berlangsung. Daftar `train_losses` yang telah dikumpulkan pada tahapan sebelumnya diplot ke dalam bentuk grafik garis menggunakan pustaka `matplotlib`. Grafik tersebut berfungsi sebagai indikator visual untuk menganalisis apakah model berhasil melakukan proses pembelajaran dengan baik, yang umumnya ditandai dengan kurva yang menurun tajam pada *epoch* awal dan berangsur stabil (konvergen) seiring dengan bertambahnya jumlah iterasi.



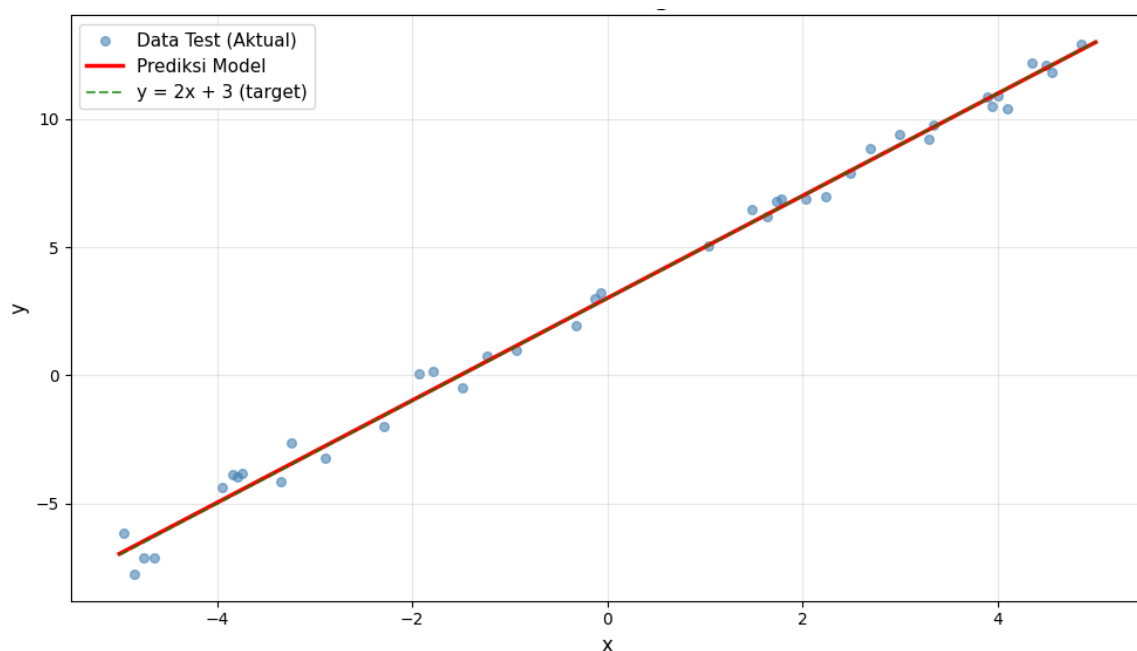
Gambar 3.4: Grafik penurunan nilai loss terhadap iterasi (*epoch*), memvisualisasikan bagaimana model belajar meminimalkan kesalahan prediksi pada data latih.

CODE

```
# =====  
# PLOT TRAINING LOSS  
# =====  
  
plt.figure(figsize=(10, 5))  
plt.plot(range(1, num_epochs+1), train_losses, color='crimson',  
linewidth=2)  
plt.xlabel('Epoch', fontsize=12)  
plt.ylabel('Loss (MSE)', fontsize=12)  
plt.title('Training Loss - Regresi Linear', fontsize=14,  
fontweight='bold')  
plt.grid(True, alpha=0.3)  
plt.tight_layout()  
plt.show()
```

3.2.G. EVALUASI MODEL DAN VISUALISASI PREDIKSI

Bagian terakhir memuat tahapan evaluasi performa model menggunakan data uji serta visualisasi akhir. Pemanggilan metode `model_regresi.eval()` mengubah pengaturan model ke mode evaluasi. Penggunaan sintaks `with torch.no_grad()`: bertugas menghentikan sementara pelacakan gradien oleh PyTorch, sehingga sangat menghemat memori dan meningkatkan efisiensi komputasi saat menguji model. Nilai *loss* dari data uji kemudian dihitung dan dicetak. Parameter bobot dan bias yang telah dipelajari juga diekstraksi, yang pada tahap ini semestinya memiliki nilai mendekati konfigurasi asli pembentuk data (bobot sekitar 2.0 dan bias sekitar 3.0). Sebagai penutup, grafik komparasi digambar untuk menampilkan persilangan antara titik-titik data uji aktual dengan garis kontinu yang merupakan hasil prediksi dari model regresi yang telah dilatih.



Gambar 3.5: Visualisasi performa akhir model pada data yang belum pernah dilihat (unseen data), menunjukkan kemampuan model dalam menangkap pola linear dari dataset yang mengandung noise.

CODE

```
# =====  
# EVALUASI PADA DATA TEST  
# =====  
  
# Mode evaluasi (menonaktifkan dropout, batch norm, dll.)  
model_regresi.eval()  
  
with torch.no_grad(): # Tidak perlu hitung gradien saat evaluasi  
    y_pred_test = model_regresi(X_test)  
    test_loss = criterion(y_pred_test, y_test)  
    print(f"Test Loss (MSE): {test_loss.item():.4f}")
```

```

# Lihat bobot yang dipelajari
for name, param in model_regresi.named_parameters():
    print(f"    {name}: {param.data.item():.4f}")
    # Seharusnya mendekati  $w \approx 2.0$  dan  $b \approx 3.0$ 

# =====
# PLOT HASIL PREDIKSI
# =====

# Prediksi untuk seluruh range X
model_regresi.eval()
with torch.no_grad():
    X_plot = torch.linspace(-5, 5, 100).reshape(-1, 1)
    y_plot = model_regresi(X_plot)

plt.figure(figsize=(10, 6))
plt.scatter(X_test.numpy(), y_test.numpy(), alpha=0.6, s=30,
            color='steelblue', label='Data Test (Aktual)')
plt.plot(X_plot.numpy(), y_plot.numpy(), 'r-', linewidth=2.5,
         label='Prediksi Model')
plt.plot(X_plot.numpy(), (2*X_plot + 3).numpy(), 'g--', linewidth=1.5,
         label='y = 2x + 3 (target)', alpha=0.7)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.title('Hasil Prediksi - Regresi Linear', fontsize=14,
         fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

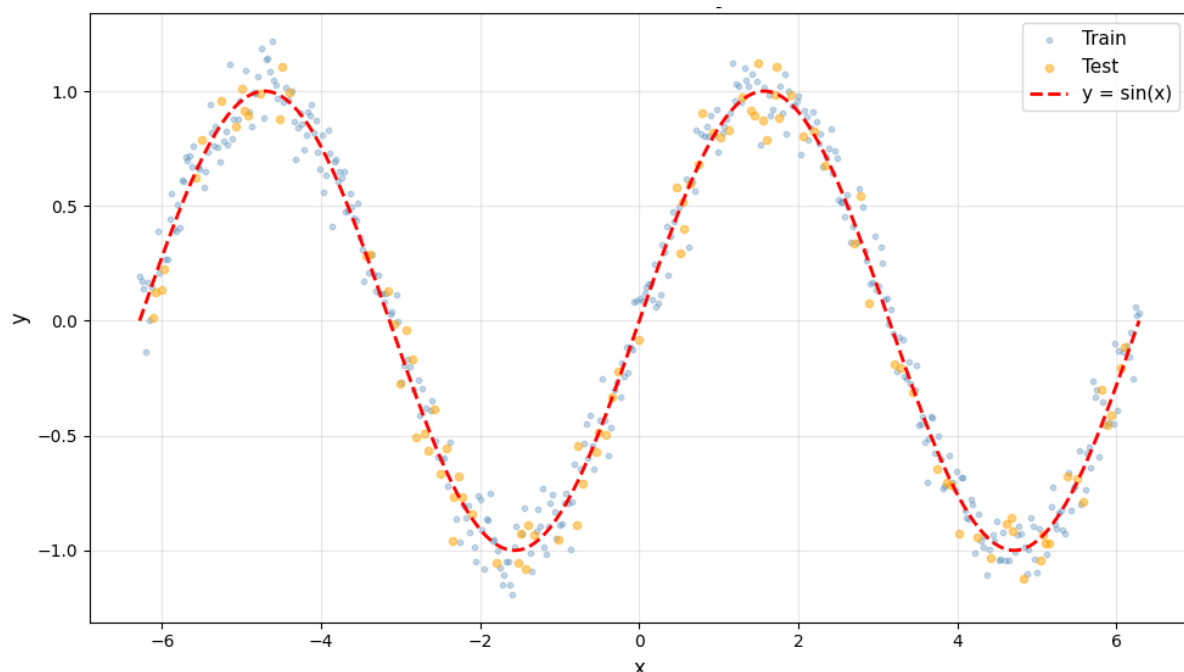
```


3.3 REGRESI NON-LINEAR (MULTI-LAYER)

3.3.A. MEMBUAT DATASET NON-LINEAR

Blok kode pertama berfokus pada persiapan awal, manipulasi matriks, dan visualisasi dataset. Langkah paling awal dimulai dengan menetapkan *seed* acak `torch.manual_seed()` agar data dan pengacakan yang dihasilkan bernilai konsisten setiap kali program dijalankan. Dataset sintetis kemudian dibuat berdasarkan fungsi trigonometri $y = \sin(x)$ menggunakan 500 titik sampel. Penambahan *noise* acak melalui `torch.randn()` dilakukan agar data tersebut memiliki sebaran yang tidak terlalu sempurna, menyerupai kondisi data pada dunia nyata.

Setelah data terbentuk, dataset dibagi secara acak dengan proporsi 80% sebagai data latih (*training*) dan 20% sebagai data uji (*testing*). Data latih tersebut dibungkus ke dalam fungsi `DataLoader` dengan ukuran *batch* 32. Fungsi ini bertugas memecah keseluruhan data latih menjadi kelompok-kelompok kecil secara acak di setiap iterasi, yang sangat berguna untuk mengoptimalkan memori komputasi dan menjaga stabilitas pembaruan bobot. Blok kode diakhiri dengan penggunaan modul Matplotlib untuk menampilkan visualisasi sebaran titik data latih dan data uji, memberikan gambaran langsung mengenai pola data non-linear yang akan diselesaikan.



Gambar 3.6: Distribusi dataset non-linear berdasarkan fungsi trigonometri $y = \sin(x)$ yang dibagi menjadi data latih (steelblue) dan data uji (orange) dengan tambahan *noise* Gaussian.

CODE

```
# =====
# DATASET: y = sin(x) + noise
# =====

torch.manual_seed(42)

n_samples = 500
X = torch.linspace(-2 * np.pi, 2 * np.pi, n_samples).reshape(-1, 1)
y_true = torch.sin(X) + torch.randn(n_samples, 1) * 0.1

# Split data
indices = torch.randperm(n_samples)
train_size = int(0.8 * n_samples)

X_train = X[indices[:train_size]]
y_train = y_true[indices[:train_size]]
X_test = X[indices[train_size:]]
y_test = y_true[indices[train_size:]]

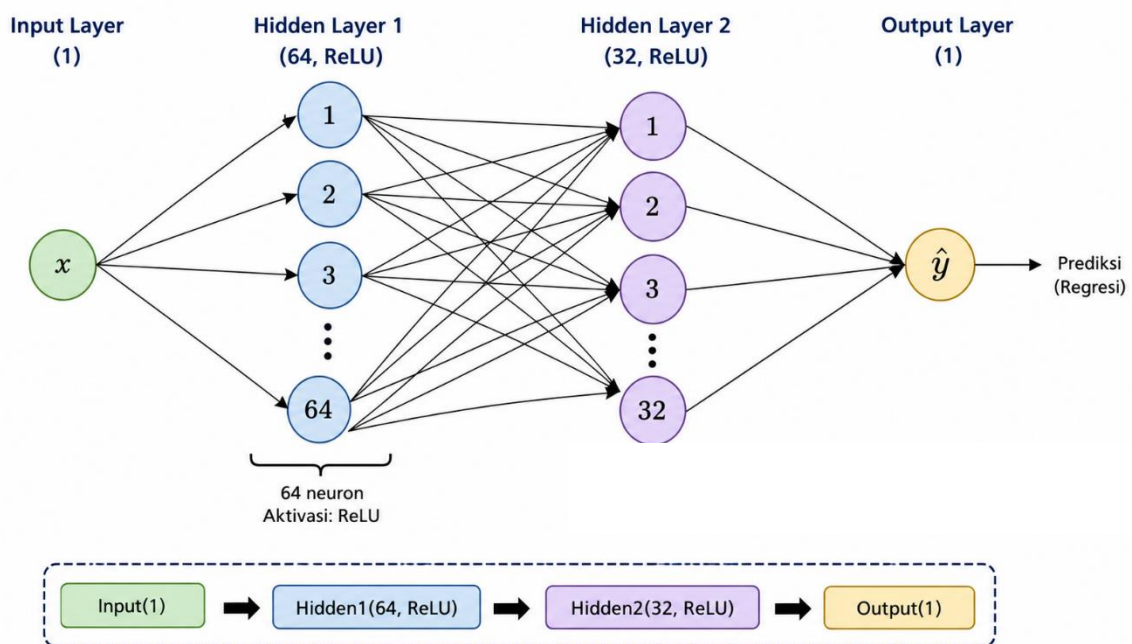
# DataLoader
train_dataset = TensorDataset(X_train, y_train)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# Visualisasi
plt.figure(figsize=(10, 6))
plt.scatter(X_train.numpy(), y_train.numpy(), alpha=0.3, s=10,
            color='steelblue', label='Train')
plt.scatter(X_test.numpy(), y_test.numpy(), alpha=0.5, s=20,
            color='orange', label='Test')
plt.plot(X.numpy(), torch.sin(X).numpy(), 'r--', linewidth=2,
         label='y = sin(x)')
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.title('Dataset Non-Linear: y = sin(x)', fontsize=14,
         fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

3.3.B. MEMBANGUN MODEL MULTI-LAYER

Blok kode kedua menjelaskan pembentukan struktur atau arsitektur *Artificial Neural Network* (ANN) yang didefinisikan menggunakan metode `nn.Sequential`. Arsitektur dirancang memiliki karakteristik jaringan yang lebih dalam (*multi-layer*) dengan menempatkan dua *hidden layer* di antara *input* dan *output*. Parameter fitur tunggal masuk ke *layer* tersembunyi pertama yang memiliki 64 *neuron*, kemudian menyempit di *layer* tersembunyi kedua menjadi 32 *neuron*.

Satu elemen yang paling krusial pada kode ini adalah penambahan fungsi aktivasi ReLU (*Rectified Linear Unit*) di setiap pergantian *hidden layer*. Tanpa ReLU, lapisan sebanyak apa pun hanya akan menghasilkan persamaan linear biasa. ReLU memberikan sifat non-linearitas yang memungkinkan jaringan belajar membentuk lengkungan pola fungsi sinus yang kompleks. Pada ujung arsitektur, *output layer* dibiarkan linear dengan 1 *neuron* tanpa fungsi aktivasi tambahan, mengingat modul ini menyelesaikan masalah regresi untuk mencari suatu nilai numerik. Bagian bawah kode secara otomatis menghitung dan menjumlahkan total parameter matematis atau bobot yang perlu diperbarui oleh sistem selama proses pembelajaran.



Gambar 3.7: Arsitektur Deep Neural Network untuk regresi non-linear yang terdiri dari dua hidden layer dengan fungsi aktivasi ReLU untuk menangkap pola data yang kompleks.

CODE

```
# =====  
# MODEL: ANN dengan Hidden Layers  
# =====  
  
# Arsitektur:  
#   Input(1) → Hidden1(64, ReLU) → Hidden2(32, ReLU) → Output(1)  
#  
#   Layer 1: 1 fitur → 64 neuron + ReLU  
#   Layer 2: 64 neuron → 32 neuron + ReLU  
#   Layer 3: 32 neuron → 1 output (tanpa aktivasi, karena regresi)  
  
model_nonlinear = nn.Sequential(  
    nn.Linear(1, 64),      # Hidden layer 1  
    nn.ReLU(),             # Aktivasi ReLU  
    nn.Linear(64, 32),     # Hidden layer 2  
    nn.ReLU(),             # Aktivasi ReLU  
    nn.Linear(32, 1)       # Output layer (tanpa aktivasi → regresi)  
)  
  
print("Arsitektur Model:")  
print(model_nonlinear)  
  
# Hitung total parameter  
total_params = sum(p.numel() for p in model_nonlinear.parameters())  
print(f"\nTotal parameter: {total_params:,}")  
# (1*64 + 64) + (64*32 + 32) + (32*1 + 1) = 128 + 2080 + 33 = 2,241
```

3.3.C. TRAINING LOOP

Blok kode ketiga merepresentasikan inti komputasi di mana model mulai dilatih untuk memahami data. Inisialisasi awal menetapkan fungsi *Mean Squared Error* (MSE) sebagai pengukur jarak antara hasil prediksi dan data aktual, dipasangkan dengan algoritma optimasi *Adam* yang bertugas menyesuaikan bobot model secara iteratif berbekal *learning rate* 0.001. Algoritma disetel untuk berulang kali membaca data dalam sebuah siklus sebanyak 200 *epoch*.

Di dalam proses perulangan, model diinstruksikan dalam mode pelatihan (`model.train()`). Untuk setiap *batch*, program melakukan *forward pass* untuk menghasilkan tebakan, menghitung kerugian dari tebakan tersebut, mengosongkan jejak gradien masa lalu melalui `zero_grad()`, dan menjalankan *backward pass* guna mencari tahu ke arah mana bobot perlu dikoreksi, sebelum akhirnya dieksekusi oleh `optimizer.step()`. Di sisi lain, blok kode ini juga menguji kehebatan model setiap kali sebuah *epoch* usai menggunakan mode evaluasi (`model.eval()`). Dengan menyertakan konteks `torch.no_grad()`, beban komputasi ditekan saat

menghitung validasi data uji karena sistem tidak diwajibkan mengingat kalkulasi gradien. Hasil *loss* evaluasi ini dicetak berkala agar tren pembelajaran dapat dipantau.

CODE

```
# =====  
# LOSS FUNCTION & OPTIMIZER  
# =====  
  
criterion = nn.MSELoss()  
optimizer = optim.Adam(model_nonlinear.parameters(), lr=0.001)  
  
# =====  
# TRAINING LOOP  
# =====  
  
num_epochs = 200  
train_losses = []  
test_losses = []  
  
print("Mulai Training Model Non-Linear...")  
print("-" * 60)  
  
for epoch in range(num_epochs):  
    # --- Training Phase ---  
    model_nonlinear.train()  
    epoch_loss = 0.0  
    num_batches = 0  
  
    for X_batch, y_batch in train_loader:  
        y_pred = model_nonlinear(X_batch)  
        loss = criterion(y_pred, y_batch)  
  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
  
        epoch_loss += loss.item()  
        num_batches += 1  
  
    avg_train_loss = epoch_loss / num_batches  
    train_losses.append(avg_train_loss)  
  
    # --- Evaluation Phase ---  
    model_nonlinear.eval()  
    with torch.no_grad():  
        y_pred_test = model_nonlinear(X_test)  
        test_loss = criterion(y_pred_test, y_test)
```



```

        test_losses.append(test_loss.item())

    # Print setiap 20 epoch
    if (epoch + 1) % 20 == 0:
        print(f"Epoch [{epoch+1:3d}/{num_epochs}] | "
              f"Train Loss: {avg_train_loss:.6f} | "
              f"Test Loss: {test_loss.item():.6f}")

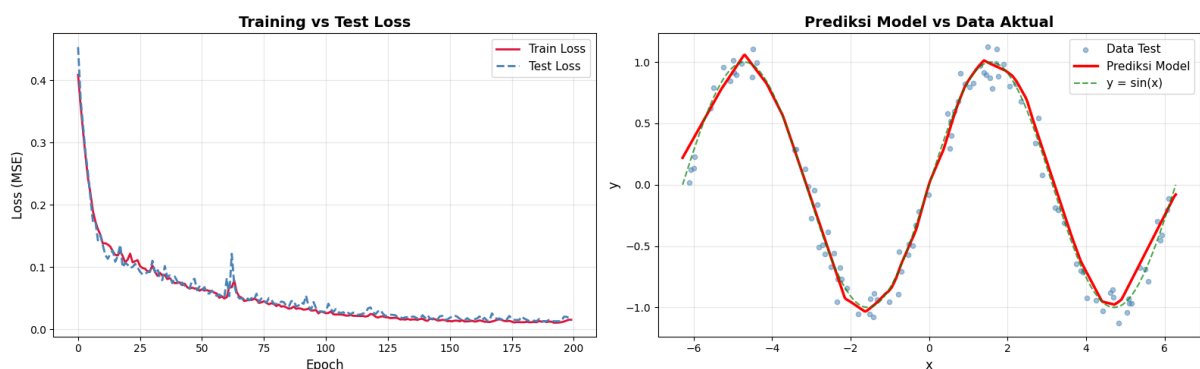
print("-" * 60)
print("Training Selesai!")

```

3.3.D. EVALUASI DAN VISUALISASI HASIL PREDIKSI

Blok kode keempat dan terakhir berfungsi sebagai penilai akhir atas kinerja model melalui penyajian grafik visual. `Matplotlib` digunakan untuk membagi ruang gambar menjadi dua panel. Panel pertama membuat kurva garis yang membandingkan tren penurunan nilai kesalahan dari data latih (*Train Loss*) dan data uji (*Test Loss*) secara bersamaan. Visualisasi pada kurva pertama ini wajib dianalisis untuk memastikan tidak terjadinya kondisi *overfitting*, yakni ketika model terlalu menghafal data latih namun gagal ketika dihadapkan pada data uji.

Pada panel grafik kedua, visualisasi difokuskan pada pengujian kemampuan adaptasi arsitektur jaringan secara keseluruhan. Garis prediksi model berwarna merah digambar membelah titik-titik data uji (*scatter plot*), dan disandingkan dengan garis aktual berbentuk fungsi $y = \sin(x)$. Melalui hasil plot kurva tersebut, keberhasilan konfigurasi dari *hidden layer* beserta aktivasi ReLU dapat diverifikasi. Jaringan yang dikonfigurasi dan dilatih secara tepat akan menunjukkan sebuah garis lengkung yang secara rapi mengikuti liukan ombak data aslinya.



Gambar 3.8: Visualisasi performa model neural network nonlinear yang menampilkan kurva Training vs Test Loss (kiri) untuk memantau konvergensi model, serta perbandingan antara Prediksi Model terhadap Data Aktual dan fungsi dasar $y = \sin(x)$ (kanan).

CODE

```
# =====
# PLOT 1: Training vs Test Loss
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Plot loss
axes[0].plot(train_losses, label='Train Loss', color='crimson',
             linewidth=2)
axes[0].plot(test_losses, label='Test Loss', color='steelblue',
             linewidth=2, linestyle='--')
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Loss (MSE)', fontsize=12)
axes[0].set_title('Training vs Test Loss', fontsize=14,
                 fontweight='bold')
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)

# =====
# PLOT 2: Prediksi vs Aktual
# =====

model_nonlinear.eval()
with torch.no_grad():
    X_plot = torch.linspace(-2*np.pi, 2*np.pi, 500).reshape(-1, 1)
    y_plot = model_nonlinear(X_plot)

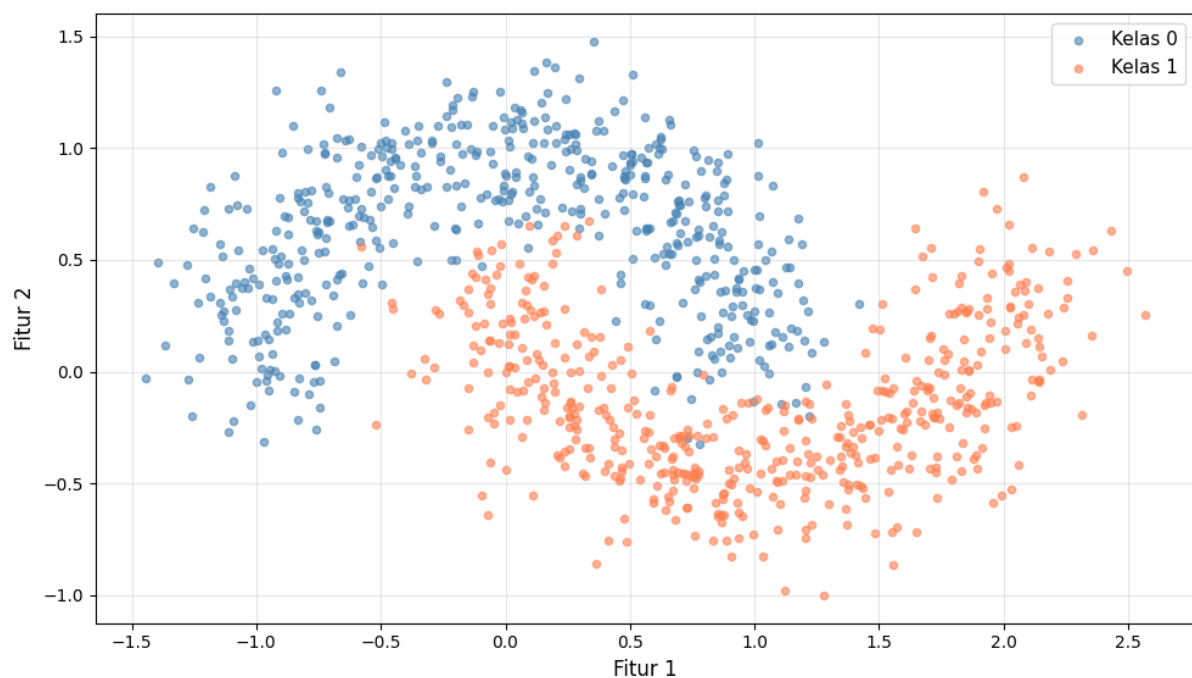
axes[1].scatter(X_test.numpy(), y_test.numpy(), alpha=0.5, s=20,
                color='steelblue', label='Data Test')
axes[1].plot(X_plot.numpy(), y_plot.numpy(), 'r-', linewidth=2.5,
             label='Prediksi Model')
axes[1].plot(X_plot.numpy(), torch.sin(X_plot).numpy(), 'g--',
             linewidth=1.5, alpha=0.7, label='y = sin(x)')
axes[1].set_xlabel('x', fontsize=12)
axes[1].set_ylabel('y', fontsize=12)
axes[1].set_title('Prediksi Model vs Data Aktual', fontsize=14,
                 fontweight='bold')
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

3.4 KLASIFIKASI BINER

3.4.A. MEMBUAT DATASET

Blok kode pertama berfokus pada tahap pembuatan dataset sintetis menggunakan fungsi `make_moons` dari pustaka `Scikit-Learn`. Dataset yang dibangkitkan terdiri atas 1000 sampel data yang terbagi secara berimbang ke dalam dua kelas berbeda, membentuk pola sebaran titik yang menyerupai dua bulan sabit yang saling berpotongan. Pemberian parameter nilai *noise* sebesar 0,2 ditujukan untuk membuat distribusi data sedikit menyebar dan tumpang tindih, meniru ketidaksempurnaan maupun gangguan (*outliers*) yang sering ditemui pada data di dunia nyata. Selanjutnya, pustaka `Matplotlib` digunakan untuk memproyeksikan data ke dalam grafik *scatter plot*. Titik-titik direpresentasikan dengan warna berbeda (biru untuk Kelas 0 dan oranye untuk Kelas 1) guna memberikan ilustrasi gamblang bahwa data ini tidak linier dan tidak dapat dipisahkan secara sempurna hanya dengan menggunakan sebuah garis lurus.



Gambar 3.9: Visualisasi sebaran data pada dataset Make Moons untuk tugas klasifikasi biner. Gambar menunjukkan dua kelompok data (Kelas 0 dan Kelas 1) yang membentuk pola non-linear melengkung dengan tingkat noise sebesar 0.2.

CODE

```
# =====  
# DATASET: Make Moons (2 kelas)  
# =====  
  
from sklearn.datasets import make_moons  
  
# Generate data berbentuk bulan sabit  
X_np, y_np = make_moons(n_samples=1000, noise=0.2, random_state=42)  
  
# Visualisasi  
plt.figure(figsize=(10, 6))  
plt.scatter(X_np[y_np == 0, 0], X_np[y_np == 0, 1],  
            c='steelblue', label='Kelas 0', alpha=0.6, s=20)  
plt.scatter(X_np[y_np == 1, 0], X_np[y_np == 1, 1],  
            c='coral', label='Kelas 1', alpha=0.6, s=20)  
plt.xlabel('Fitur 1', fontsize=12)  
plt.ylabel('Fitur 2', fontsize=12)  
plt.title('Dataset Make Moons - Klasifikasi Biner', fontsize=14,  
          fontweight='bold')  
plt.legend(fontsize=11)  
plt.grid(True, alpha=0.3)  
plt.tight_layout()  
plt.show()
```

3.4.B. PREPROCESSING & SPLIT DATA

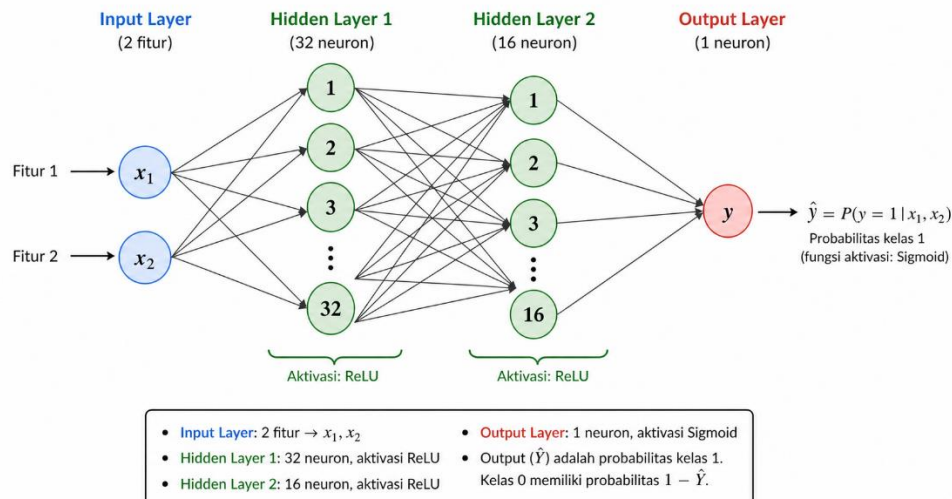
Blok kode kedua memuat langkah-langkah prapemrosesan agar struktur data sesuai dengan kebutuhan kerangka kerja PyTorch. Proses dimulai dengan standarisasi fitur melalui objek `StandardScaler`. Tahap ini mengonversi rentang nilai fitur agar memiliki nilai rata-rata nol dan simpangan baku satu, yang berfungsi krusial untuk mencegah dominasi suatu fitur berskala besar dan mempercepat stabilitas konvergensi model. Data yang telah diskalakan kemudian dipartisi menjadi kumpulan data latih (80%) dan data uji (20%). Mengingat model beroperasi di lingkungan PyTorch, seluruh *array* berformat NumPy dialihbentukkan menjadi struktur *tensor* 32-bit (FloatTensor). Dimensi label sasaran juga dibentuk ulang agar berupa matriks berkolom tunggal. Pada akhir blok, data latih dibungkus menggunakan antarmuka `DataLoader` untuk dipecah menjadi kelompok-kelompok kecil (*batch*) berisi 32 sampel, yang memungkinkan komputasi iteratif secara lebih efisien dan hemat memori saat model dilatih.

CODE

```
# =====  
# PREPROCESSING  
# =====  
  
# Standardisasi fitur (mean=0, std=1)  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_np)  
  
# Split data  
X_train_np, X_test_np, y_train_np, y_test_np = train_test_split(  
    X_scaled, y_np, test_size=0.2, random_state=42  
)  
  
# Konversi ke tensor PyTorch  
X_train = torch.FloatTensor(X_train_np)  
y_train = torch.FloatTensor(y_train_np).reshape(-1, 1) # Shape: (N,  
1)  
X_test = torch.FloatTensor(X_test_np)  
y_test = torch.FloatTensor(y_test_np).reshape(-1, 1)  
  
print(f"X_train shape: {X_train.shape}") # (800, 2)  
print(f"y_train shape: {y_train.shape}") # (800, 1)  
print(f"X_test shape : {X_test.shape}") # (200, 2)  
print(f"y_test shape : {y_test.shape}") # (200, 1)  
  
# DataLoader  
train_dataset = TensorDataset(X_train, y_train)  
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
```

3.4.C. MEMBANGUN MODEL

Blok kode ketiga berfungsi mendefinisikan bentuk dan kedalaman arsitektur *Artificial Neural Network* (ANN) dengan mengaplikasikan metode `nn.Sequential`. Arsitektur dirancang untuk menerima masukan berupa dua fitur dari *dataset*, yang kemudian mengalir melewati dua lapisan tersembunyi (*hidden layer*) berukuran 32 dan 16 *neuron*. Setiap *hidden layer* dirangkaikan dengan fungsi aktivasi ReLU (*Rectified Linear Unit*). Keterlibatan ReLU sangat esensial untuk memperkenalkan perhitungan non-linearitas, yang mengizinkan sistem untuk beradaptasi dan mempelajari lekukan kurva dari pola bulan sabit tersebut. Pada ujung struktur, lapisan *output* menyempit menjadi satu *neuron* tunggal yang diikat dengan fungsi aktivasi *Sigmoid*. Fungsi *Sigmoid* bertugas memampatkan nilai mentah komputasi menjadi format probabilitas berentang 0 hingga 1, yang akan diterjemahkan sebagai tingkat keyakinan prediktif model terhadap kelas positif (Kelas 1).



Gambar 3.10: Arsitektur jaringan saraf tiruan untuk klasifikasi biner yang terdiri dari satu lapis input dengan 2 fitur, dua hidden layer masing-masing dengan 32 dan 16 neuron (ReLU), serta satu lapis output dengan fungsi aktivasi Sigmoid untuk menghasilkan probabilitas kelas.

CODE

```
# =====
# MODEL: Klasifikasi Biner
# =====

# Arsitektur:
#   Input(2) → Hidden1(32, ReLU) → Hidden2(16, ReLU) → Output(1, Sigmoid)
#
#   - Input: 2 fitur ( $x_1, x_2$ )
#   - Hidden 1: 32 neuron dengan ReLU
#   - Hidden 2: 16 neuron dengan ReLU
#   - Output: 1 neuron dengan Sigmoid → probabilitas kelas 1

model_binary = nn.Sequential(
    nn.Linear(2, 32),      # Input → Hidden 1
    nn.ReLU(),
    nn.Linear(32, 16),     # Hidden 1 → Hidden 2
    nn.ReLU(),
    nn.Linear(16, 1),      # Hidden 2 → Output
    nn.Sigmoid()           # Aktivasi Sigmoid → output antara 0-1
)

print("Arsitektur Model:")
print(model_binary)
total_params = sum(p.numel() for p in model_binary.parameters())
print(f"\nTotal parameter: {total_params:,}")
```


3.4.D. LOSS FUNCTION, OPTIMASI, DAN TRAINING LOOP

Blok kode keempat merupakan pusat komputasi matematis tempat pembelajaran iteratif (*training loop*) dijalankan. Persiapan diawali dengan menentukan *Binary Cross Entropy* (BCELoss) sebagai alat ukur tingkat kesalahan model untuk klasifikasi dua kelas, bersanding dengan algoritma optimasi *Adam* yang mengendalikan proses koreksi bobot jaringan dengan laju pembelajaran (*learning rate*) 0,001. Siklus pelatihan diprogram untuk berputar sebanyak 100 *epoch*. Di dalam setiap iterasinya, model meninjau *batch* data, memprediksi probabilitas keluaran (*forward pass*), menakar besaran eror dari prediksi tersebut, merambatkan nilai kesalahan ke belakang layar (*backward pass*), hingga akhirnya melakukan penyesuaian parameter secara mandiri. Evaluasi performa melalui data uji juga disisipkan tanpa merekam jejak kalkulasi gradien (`torch.no_grad()`). Ambang batas (*threshold*) sebesar 0,5 diterapkan untuk menentukan kategori kelas secara tegas dari rentang probabilitas yang dikembalikan oleh jaringan.

CODE

```
# =====
# LOSS & OPTIMIZER
# =====

# BCELoss karena output sudah melalui Sigmoid
criterion = nn.BCELoss()
optimizer = optim.Adam(model_binary.parameters(), lr=0.001)

# =====
# TRAINING LOOP
# =====

num_epochs = 100
train_losses = []
train_accs = []
test_accs = []

print("Mulai Training Klasifikasi Biner...")
print("-" * 65)

for epoch in range(num_epochs):
    model_binary.train()
    epoch_loss = 0.0
    correct = 0
    total = 0
```

```

for X_batch, y_batch in train_loader:
    # Forward pass
    y_pred = model_binary(X_batch)

    # Hitung loss
    loss = criterion(y_pred, y_batch)

    # Backward pass & update
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    epoch_loss += loss.item()

    # Hitung akurasi (threshold = 0.5)
    predicted = (y_pred >= 0.5).float()
    correct += (predicted == y_batch).sum().item()
    total += y_batch.size(0)

avg_loss = epoch_loss / len(train_loader)
train_acc = correct / total * 100
train_losses.append(avg_loss)
train_accs.append(train_acc)

# Evaluasi pada data test
model_binary.eval()
with torch.no_grad():
    y_pred_test = model_binary(X_test)
    predicted_test = (y_pred_test >= 0.5).float()
    test_acc = (predicted_test == y_test).sum().item() /
y_test.size(0) * 100
    test_accs.append(test_acc)

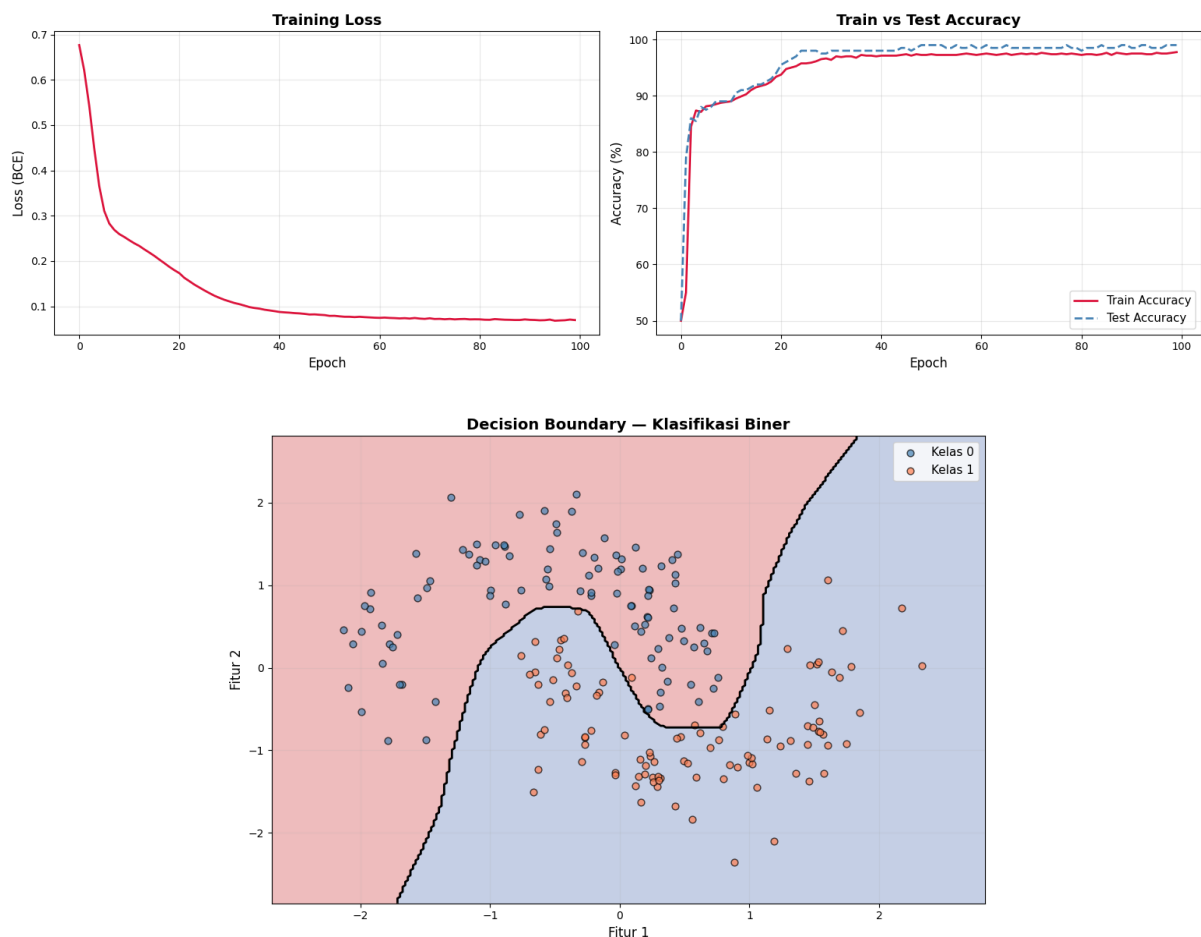
if (epoch + 1) % 10 == 0:
    print(f"Epoch [{epoch+1:3d}/{num_epochs}] | "
          f"Loss: {avg_loss:.4f} | "
          f"Train Acc: {train_acc:.1f}% | "
          f"Test Acc: {test_acc:.1f}%")

print("-" * 65)
print("Training Selesai!")
print(f"\nAkurasi Akhir → Train: {train_accs[-1]:.1f}% | Test:
{test_accs[-1]:.1f}%")

```

3.4.E. VISUALISASI KINERJA DAN BATAS KEPUTUSAN

Blok kode kelima dijalankan untuk membuktikan keandalan jaringan saraf pascapelatihan melalui panel antarmuka visual. Grafis perbandingan pertama menyuguhkan kurva linear yang menelusuri fluktuasi grafik kesalahan (*loss*) dan tingkat akurasi untuk observasi uji coba seiring berjalannya setiap *epoch*. Grafik lintasan tersebut berguna untuk mengkonfirmasi bahwa proses pembelajaran berjalan secara wajar tanpa terperangkap ke dalam masalah *overfitting*. Pada antarmuka kedua, algoritma menyusun kisi-kisi titik padat (*meshgrid*) di seantero koordinat ruang fitur, dan melempar keseluruhan titik tersebut ke dalam model untuk menebak asal kelasnya. Peta area dipetakan dan dikonturkan dengan dua wilayah warna yang dipecah oleh sebuah garis patahan. Tampilan ini, yang dikenal sebagai *Decision Boundary* atau batas keputusan, mendemonstrasikan secara transparan kebolehan batas adaptif yang dibangun oleh model saraf tiruan dalam mengisolasi kluster data berpolaritas Kelas 0 dan Kelas 1.



Gambar 3.11: Evaluasi performa model klasifikasi biner pada dataset Make Moons. Panel atas menunjukkan tren penurunan Binary Cross-Entropy (BCE) Loss dan peningkatan akurasi pada data latih serta uji. Panel bawah menunjukkan decision boundary non-linear yang dihasilkan model dalam memisahkan Kelas 0 dan Kelas 1 pada ruang fitur.

CODE

```
# =====
# PLOT 1: Loss & Accuracy
# =====

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Loss
axes[0].plot(train_losses, color='crimson', linewidth=2)
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Loss (BCE)', fontsize=12)
axes[0].set_title('Training Loss', fontsize=14, fontweight='bold')
axes[0].grid(True, alpha=0.3)

# Accuracy
axes[1].plot(train_accs, label='Train Accuracy', color='crimson',
             linewidth=2)
axes[1].plot(test_accs, label='Test Accuracy', color='steelblue',
             linewidth=2, linestyle='--')
axes[1].set_xlabel('Epoch', fontsize=12)
axes[1].set_ylabel('Accuracy (%)', fontsize=12)
axes[1].set_title('Train vs Test Accuracy', fontsize=14,
                 fontweight='bold')
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# =====
# PLOT 2: Decision Boundary
# =====

# Buat grid untuk decision boundary
h = 0.02 # step size
x_min, x_max = X_scaled[:, 0].min() - 0.5, X_scaled[:, 0].max() + 0.5
y_min, y_max = X_scaled[:, 1].min() - 0.5, X_scaled[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                    np.arange(y_min, y_max, h))

# Prediksi untuk setiap titik di grid
grid_tensor = torch.FloatTensor(np.c_[xx.ravel(), yy.ravel()])
model_binary.eval()
with torch.no_grad():
    Z = model_binary(grid_tensor)
    Z = (Z >= 0.5).float().numpy().reshape(xx.shape)
```

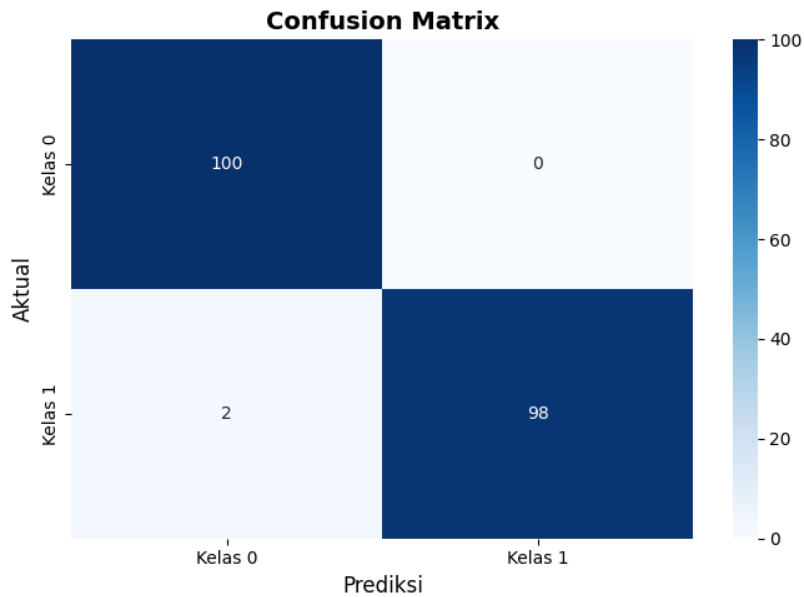
```

plt.figure(figsize=(10, 7))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='RdYlBu')
plt.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2)
plt.scatter(X_test_np[y_test_np == 0, 0], X_test_np[y_test_np == 0,
1],
            c='steelblue', label='Kelas 0', edgecolors='k', s=40,
alpha=0.7)
plt.scatter(X_test_np[y_test_np == 1, 0], X_test_np[y_test_np == 1,
1],
            c='coral', label='Kelas 1', edgecolors='k', s=40,
alpha=0.7)
plt.xlabel('Fitur 1', fontsize=12)
plt.ylabel('Fitur 2', fontsize=12)
plt.title('Decision Boundary – Klasifikasi Biner',
        fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.2)
plt.tight_layout()
plt.show()

```

3.4.F. CONFUSION MATRIX DAN LAPORAN KLASIFIKASI

Blok kode keenam beroperasi sebagai rincian evaluasi diagnostik terakhir dengan mengukur metrik metrik keandalan secara definitif. *Tensor* keluaran data uji dikelompokkan menjadi bilangan bulat biner murni, untuk selanjutnya dipertemukan dengan kelas data yang sebenarnya. Hubungan ini divisualkan melalui format *Confusion Matrix* berwujud matriks peta panas (*heatmap*). Tabulasi piksel warna dalam metrik matriks tersebut secara persis menjabarkan frekuensi ketepatan dan melesetnya prediksi atas dasar *True Positives*, *True Negatives*, *False Positives*, dan *False Negatives*. Di luar sajian peta panas, modul mengeksekusi cetakan tabel *Classification Report* yang mempresentasikan angka rinci di balik skor *precision*, *recall*, serta *f1-score*, yang menyoroti potret keakuratan algoritma klasifikasi secara lebih objektif dan saintifik untuk tiap-tiap kelas individu.



Gambar 3.12: Confusion Matrix hasil evaluasi model pada data pengujian. Matriks ini menunjukkan jumlah frekuensi prediksi benar (True Positive dan True Negative) serta kesalahan klasifikasi (Type I dan Type II Error) untuk Kelas 0 dan Kelas 1.

CODE

```
# =====
# CONFUSION MATRIX
# =====

from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

model_binary.eval()
with torch.no_grad():
    y_pred_test = model_binary(X_test)
    y_pred_labels = (y_pred_test >= 0.5).int().numpy().flatten()
    y_true_labels = y_test.int().numpy().flatten()

# Confusion Matrix
cm = confusion_matrix(y_true_labels, y_pred_labels)

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Kelas 0', 'Kelas 1'],
            yticklabels=['Kelas 0', 'Kelas 1'])
plt.xlabel('Prediksi', fontsize=12)
plt.ylabel('Aktual', fontsize=12)
plt.title('Confusion Matrix', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

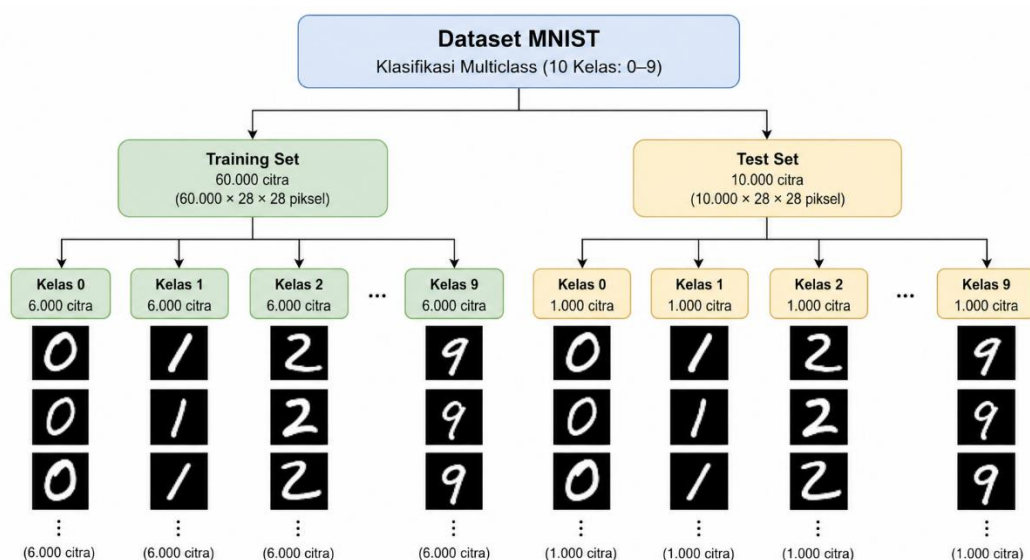


```
# Classification Report
print("\nClassification Report:")
print("=" * 55)
print(classification_report(y_true_labels, y_pred_labels,
                           target_names=['Kelas 0', 'Kelas 1']))
```

3.5 KLASIFIKASI MULTI-CLASS (MNIST)

3.5.A. MENGUNDUH DAN MEMPERSIAPKAN DATASET

Blok kode pertama berfungsi untuk mengunduh dan mempersiapkan dataset gambar angka tulisan tangan (MNIST). Proses prapemrosesan diterapkan menggunakan modul transforms, di mana gambar asli diubah menjadi format matriks angka berdimensi (*tensor*) agar dapat diproses oleh algoritma PyTorch. Selain itu, dilakukan tahapan normalisasi menggunakan nilai rata-rata dan simpangan baku spesifik yang umum digunakan untuk dataset MNIST. Hal ini bertujuan agar rentang nilai piksel gambar menjadi seimbang dan tersentris, yang akan mempercepat dan menstabilkan konvergensi saat proses pelatihan. Setelah pengaturan transformasi didefinisikan, himpunan data latih dan data uji diunduh secara otomatis dan disimpan pada direktori lokal komputer.



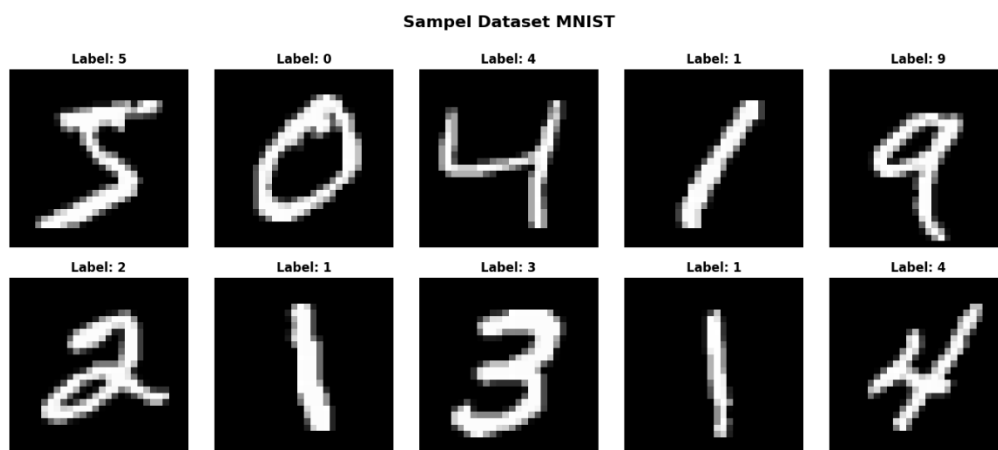
Gambar 3.13: Struktur dataset MNIST untuk klasifikasi multiclass (10 kelas)

CODE

```
# =====  
# DOWNLOAD DATASET MNIST  
# =====  
  
from torchvision import datasets, transforms  
  
# Transform: konversi gambar ke tensor & normalisasi  
transform = transforms.Compose([  
    transforms.ToTensor(),          # Konversi ke tensor (0-  
1)  
    transforms.Normalize((0.1307,), (0.3081,)) # Normalisasi (mean,  
std)  
)  
  
# Download dataset  
train_dataset = datasets.MNIST(root='./data', train=True,  
                                download=True, transform=transform)  
test_dataset = datasets.MNIST(root='./data', train=False,  
                               download=True, transform=transform)  
  
print(f"Jumlah data training: {len(train_dataset)}")  
print(f"Jumlah data testing : {len(test_dataset)}")  
print(f"Ukuran gambar      : {train_dataset[0][0].shape}") # (1, 28,  
28)  
print(f"Jumlah kelas       : 10 (digit 0-9)")
```

3.5.B. VISUALISASI SAMPEL DATASET

Blok kode kedua bertujuan untuk menampilkan cuplikan data gambar dari dataset yang telah diunduh menggunakan antarmuka `Matplotlib`. Program mengambil sepuluh sampel pertama dari dataset latihan beserta label kelas aslinya. Gambar-gambar tersebut kemudian diproyeksikan dalam bentuk kisi-kisi (*grid*) berukuran 2x5 dengan format pewarnaan skala abu-abu (*grayscale*). Visualisasi ini berperan penting untuk memverifikasi secara langsung bahwa data masukan telah diformat dan dipasangkan dengan label kebenaran (*ground truth*) yang tepat sebelum disalurkan ke dalam jaringan saraf tiruan.



Gambar 3.14: Representasi visual sepuluh sampel pertama dari dataset MNIST dalam format skala abu-abu (*grayscale*). Setiap sub-plot menampilkan citra berukuran 28×28 piksel beserta label numerik yang sesuai.

CODE

```
# =====  
# VISUALISASI SAMPEL MNIST  
# =====  
  
fig, axes = plt.subplots(2, 5, figsize=(14, 6))  
fig.suptitle('Sampel Dataset MNIST', fontsize=16, fontweight='bold',  
y=1.02)  
  
for i, ax in enumerate(axes.flat):  
    image, label = train_dataset[i]  
    ax.imshow(image.squeeze(), cmap='gray')  
    ax.set_title(f'Label: {label}', fontsize=12, fontweight='bold')  
    ax.axis('off')  
  
plt.tight_layout(h_pad=3.0)  
plt.show()
```

3.5.C. PEMBENTUKAN DATALOADER

Blok kode ketiga mengimplementasikan fungsi `DataLoader` untuk membungkus himpunan data ke dalam struktur yang dapat diiterasi. Dataset tidak diproses sekaligus secara utuh, melainkan dipecah menjadi kelompok-kelompok kecil (*batch*) yang masing-masing memuat 64 gambar. Pemecahan ukuran *batch* ini sangat esensial untuk mengoptimalkan penggunaan memori pada perangkat keras dan menstabilkan perbaruan bobot matematis jaringan. Pengacakan posisi data (*shuffle*) diaktifkan khusus untuk data latih guna mencegah model menghafal urutan masuknya gambar, sementara data uji dibiarkan berurutan sesuai aslinya.

CODE

```
# =====
# DATALOADER
# =====

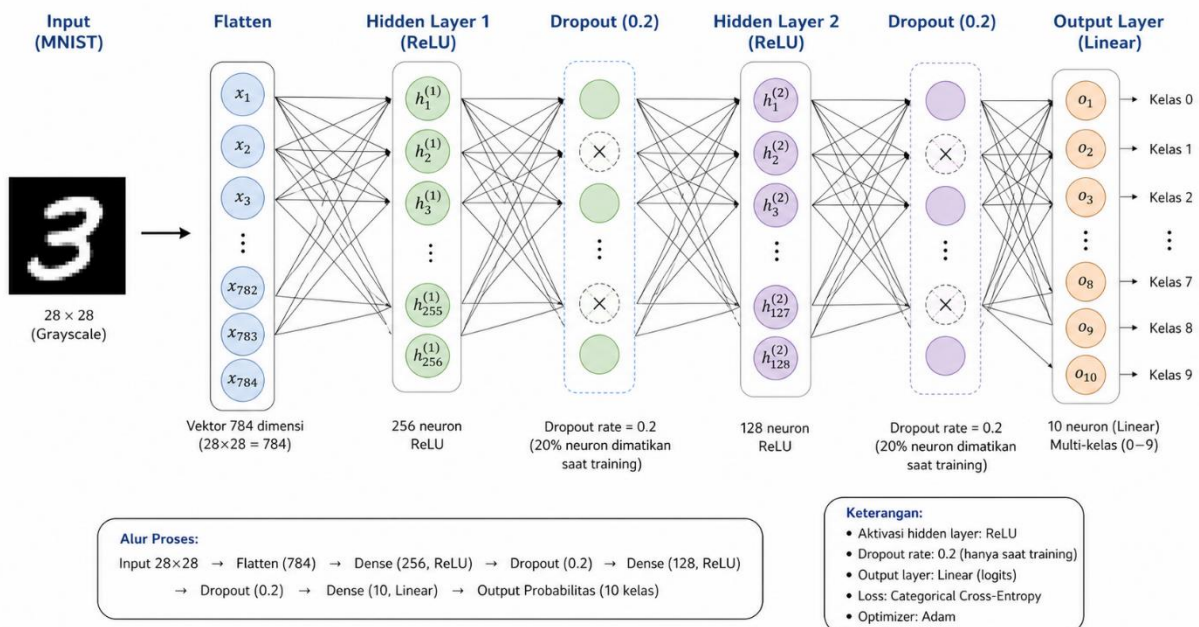
batch_size = 64

train_loader = DataLoader(train_dataset, batch_size=batch_size,
                           shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
                          shuffle=False)

# Cek 1 batch
images, labels = next(iter(train_loader))
print(f"Batch images shape: {images.shape}")    # (64, 1, 28, 28)
print(f"Batch labels shape: {labels.shape}")    # (64,)
```

3.5.D. ARSITEKTUR MODEL MULTI-KELAS

Blok kode keempat mendefinisikan arsitektur jaringan saraf tiruan utuh untuk menangani klasifikasi sepuluh kelas probabilitas. Dimensi gambar 28×28 piksel terlebih dahulu diratakan menggunakan perintah *Flatten* menjadi susunan linier berjumlah 784 elemen. Susunan tersebut diteruskan melintasi dua lapisan tersembunyi yang masing-masing berkapasitas 256 dan 128 unit komputasi, dilengkapi dengan fungsi aktivasi ReLU untuk menyaring pola fitur secara non-linear. Mekanisme regularisasi bernama *Dropout* disisipkan dengan proporsi 20% pada tiap lapisan tersembunyi untuk menonaktifkan unit secara acak, yang sangat efektif dalam menekan risiko *overfitting*. Arsitektur ini berujung pada 10 unit keluaran tanpa menyertakan fungsi aktivasi *Softmax*, karena kerangka PyTorch nantinya akan mengintegrasikan komputasi tersebut langsung ke dalam fungsi kerugian.



Gambar 3.15: Arsitektur jaringan saraf tiruan feed-forward untuk klasifikasi multi-kelas pada dataset MNIST. Model menggunakan lapisan Flatten untuk mengubah input 2D menjadi vektor 784 dimensi, dua hidden layer dengan aktivasi ReLU dan Dropout (0.2) untuk regularisasi, serta lapisan output linear dengan 10 neuron.

CODE

```
# =====
# MODEL: Klasifikasi Multi-Kelas
# =====

# MNIST: gambar 28x28 = 784 piksel → input layer memiliki 784 neuron
# Output: 10 kelas (digit 0-9)
#
# Arsitektur:
#   Flatten(28x28→784) → Hidden1(256, ReLU) → Dropout(0.2)
#                               → Hidden2(128, ReLU) → Dropout(0.2)
#                               → Output(10)
#
# CATATAN: Kita TIDAK menggunakan Softmax di output karena
#           CrossEntropyLoss di PyTorch sudah termasuk Softmax di
#           dalamnya!

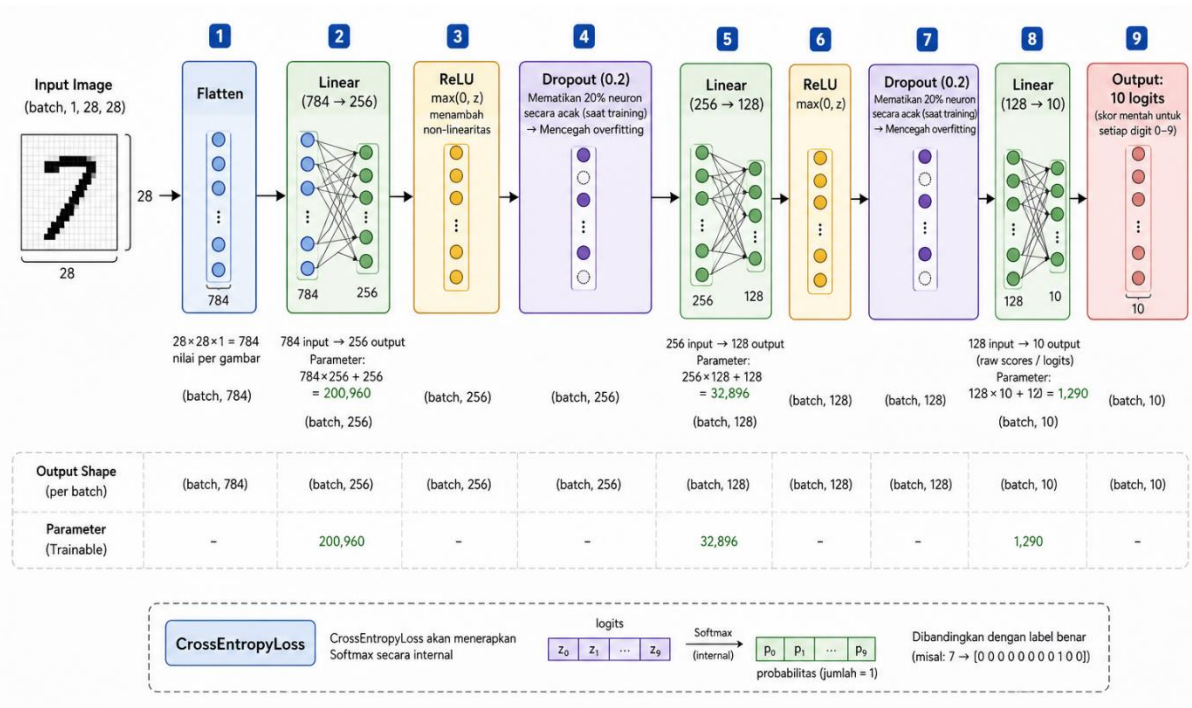
model_mnist = nn.Sequential(
    nn.Flatten(),          # (batch, 1, 28, 28) → (batch, 784)
    nn.Linear(784, 256),   # Hidden layer 1
    nn.ReLU(),
    nn.Dropout(0.2),       # Dropout 20% untuk mengurangi
overfitting
    nn.Linear(256, 128),   # Hidden layer 2
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 10)     # Output layer (10 kelas, TANPA Softmax)
)

print("Arsitektur Model MNIST:")
print(model_mnist)

total_params = sum(p.numel() for p in model_mnist.parameters())
print(f"\nTotal parameter: {total_params:,}")
# (784*256+256) + (256*128+128) + (128*10+10) = 200,960 + 32,896 +
1,290 = 235,146
```


3.5.E. DIAGRAM ALIRAN DATA

Blok diagram kelima menyajikan ilustrasi visual berbasis teks mengenai pergerakan data (*data flow*) dari titik awal masukan hingga berwujud prediksi akhir. Ilustrasi ini merangkum transformasi ukuran matriks, mulai dari wujud gambar utuh yang dipipihkan, hingga proses komputasi pada masing-masing lapisan *Linear* yang melipatgandakan serta memangkas besaran parameter jaringan. Penjabaran diagram ini membantu memvisualisasikan jumlah persis kalkulasi bobot yang diproses oleh model pada tiap tahapan perpindahan lapisan.



Gambar 3.16: Aliran data input (MNIST) melalui lapisan linear dan berpindah lapisan disetiap tahapannya sampai dengan memprediksi output (digit 0 – 9)

3.5.F. OPTIMASI DAN SIKLUS PELATIHAN

Blok kode keenam menggerakkan siklus pelatihan dan pengujian model yang diulang-ulang sebanyak sepuluh putaran (*epoch*). Komponen pengarah pembelajaran dibangun dari fungsi *Cross Entropy Loss*, yang secara internal mumpuni dalam menakar ketepatan skor *logits* untuk klasifikasi kelas ganda, dipadukan dengan optimasi Adam yang mengelola koreksi gradien model. Selama fase pelatihan, program menghitung tebakan kelas, menaksir jumlah eror, merambatkan rute galat kembali ke belakang (*backward pass*), dan memperbarui titik beban jaringan. Pada penghujung setiap putaran iteratif, performa dievaluasi menggunakan mode `eval()` tanpa merekam jejak riwayat gradien guna mempresentasikan persentase taksiran akurasi.

CODE

```
# =====  
# LOSS & OPTIMIZER  
# =====  
  
criterion = nn.CrossEntropyLoss() # Sudah termasuk Softmax!  
optimizer = optim.Adam(model_mnist.parameters(), lr=0.001)  
  
# Pindahkan model ke device (GPU jika tersedia)  
model_mnist = model_mnist.to(device)  
  
# =====  
# TRAINING LOOP  
# =====  
  
num_epochs = 10  
train_losses = []  
test_losses = []  
train_accs = []  
test_accs = []  
  
print("Mulai Training MNIST...")  
print("=" * 70)  
  
for epoch in range(num_epochs):  
    # ---- TRAINING PHASE ----  
    model_mnist.train()  
    running_loss = 0.0  
    correct = 0  
    total = 0  
  
    for images, labels in train_loader:  
        images = images.to(device)  
        labels = labels.to(device)  
  
        # Forward pass  
        outputs = model_mnist(images)  
        loss = criterion(outputs, labels)  
  
        # Backward pass & update  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
  
        running_loss += loss.item()  
  
    # Hitung akurasi
```

```

        _, predicted = torch.max(outputs, 1)  # Ambil indeks dengan
        skor tertinggi
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    train_loss = running_loss / len(train_loader)
    train_acc = correct / total * 100
    train_losses.append(train_loss)
    train_accs.append(train_acc)

    # ---- TESTING PHASE ----
    model_mnist.eval()
    test_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model_mnist(images)
            loss = criterion(outputs, labels)

            test_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    test_loss = test_loss / len(test_loader)
    test_acc = correct / total * 100
    test_losses.append(test_loss)
    test_accs.append(test_acc)

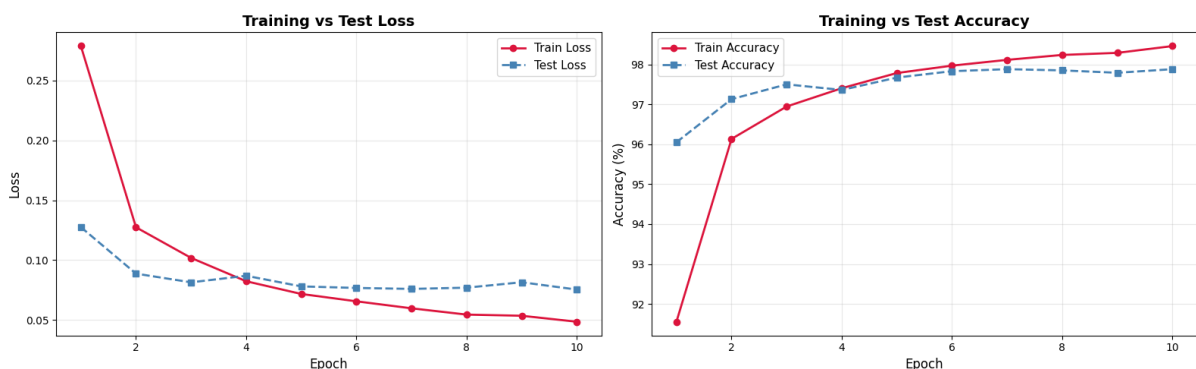
    print(f"Epoch [{epoch+1:2d}/{num_epochs}] | "
          f"Train Loss: {train_loss:.4f} | Train Acc: "
          f"{train_acc:.2f}% | "
          f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.2f}%")

    print("=" * 70)
    print(f"Akurasi Akhir → Train: {train_accs[-1]:.2f}% | Test: "
          f"{test_accs[-1]:.2f}%")

```

3.5.G. PLOT PERBANDINGAN PERFORMA

Blok kode ketujuh memvisualisasikan tren proses pembelajaran jaringan melalui penciptaan dua panel grafik garis. Titik koordinat memetakan kurva penurunan tingkat kesalahan (*loss*) dan eskalasi persentase ketepatan (*accuracy*) berdampingan di sepanjang lintasan siklus *epoch*. Penyatuan kurva data latih dan uji dalam sumbu metrik yang bersesuaian difungsikan sebagai sarana diagnosis visual bagi pemrogram untuk menilai ada tidaknya ketimpangan pada pembelajaran adaptif jaringan, sekaligus menyimpulkan stabilitas dari kecerdasan prediktif yang didapatkan.



Gambar 3.17: Perbandingan metrik evaluasi antara data latih (*training*) dan data uji (*test*) selama proses pembelajaran. Panel kiri menampilkan kurva konvergensi nilai *loss*, sementara panel kanan menunjukkan peningkatan persentase akurasi model terhadap jumlah *epoch*.

CODE

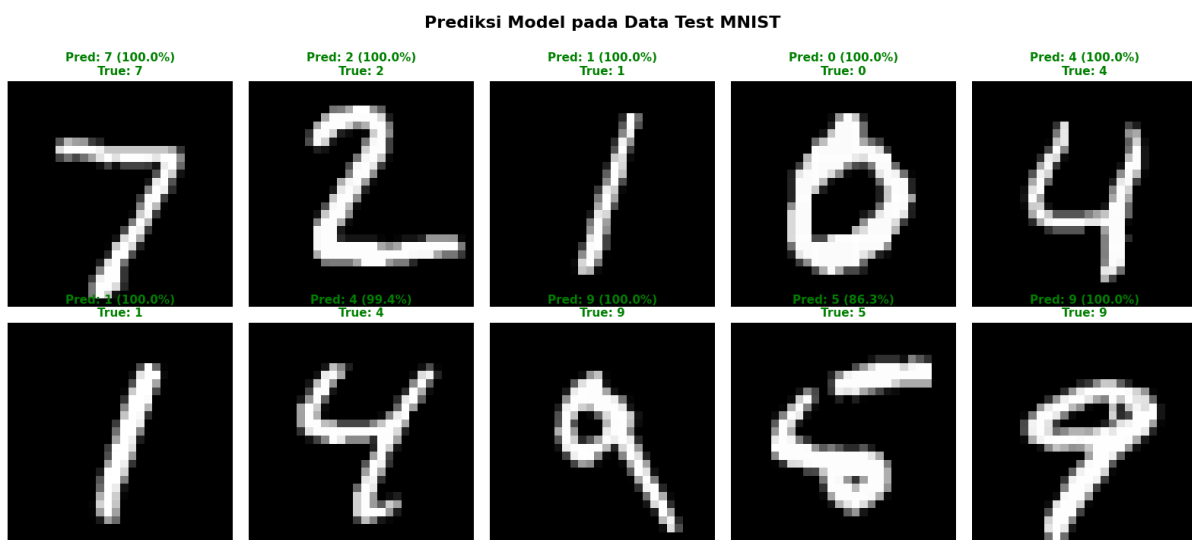
```
# =====  
# PLOT LOSS & ACCURACY  
# =====  
  
fig, axes = plt.subplots(1, 2, figsize=(16, 5))  
  
# Loss  
axes[0].plot(range(1, num_epochs+1), train_losses, 'o-',  
              color='crimson', linewidth=2, label='Train Loss')  
axes[0].plot(range(1, num_epochs+1), test_losses, 's--',  
              color='steelblue', linewidth=2, label='Test Loss')  
axes[0].set_xlabel('Epoch', fontsize=12)  
axes[0].set_ylabel('Loss', fontsize=12)  
axes[0].set_title('Training vs Test Loss', fontsize=14,  
                  fontweight='bold')  
axes[0].legend(fontsize=11)  
axes[0].grid(True, alpha=0.3)
```

```
# Accuracy
axes[1].plot(range(1, num_epochs+1), train_accs, 'o-',
             color='crimson', linewidth=2, label='Train Accuracy')
axes[1].plot(range(1, num_epochs+1), test_accs, 's--',
             color='steelblue', linewidth=2, label='Test Accuracy')
axes[1].set_xlabel('Epoch', fontsize=12)
axes[1].set_ylabel('Accuracy (%)', fontsize=12)
axes[1].set_title('Training vs Test Accuracy', fontsize=14,
                 fontweight='bold')
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

3.5.H. TINJAUAN VISUALISASI HASIL PREDIKSI

Blok kode kedelapan menjalankan simulasi pengujian kasat mata terhadap probabilitas keputusan model. Sejumlah citra yang ditarik dari bingkisan data uji diserahkan ke sistem untuk diekstrak prediksi tebakannya. Di sini, fungsi ekuasi *Softmax* diaplikasikan manual untuk mendistribusikan simpulan *logits* menjadi takaran ukuran persentase tingkat keyakinan (*confidence level*). Luaran kemudian dicetak bersama visualisasi citranya; deskripsi persentase prediksi diberikan tinta hijau apabila sejalan dengan label asli, namun dicoret dengan font berwarna merah bila asumsi dari mesin keliru secara teknis.



Gambar 3.18: Hasil inferensi model pada sampel data pengujian MNIST. Setiap gambar menampilkan perbandingan antara label prediksi dan label aktual (true label), disertai dengan tingkat kepercayaan (confidence score) dalam bentuk persentase. Indikator warna hijau menunjukkan prediksi benar, sementara warna merah (jika ada) menunjukkan kesalahan klasifikasi.

CODE

```
# =====  
# VISUALISASI PREDIKSI  
# =====  
  
model_mnist.eval()  
  
# Ambil beberapa sampel test  
test_images, test_labels = next(iter(test_loader))  
test_images_device = test_images.to(device)  
  
with torch.no_grad():  
    outputs = model_mnist(test_images_device)  
    probabilities = torch.softmax(outputs, dim=1) # Konversi logits →  
probabilitas  
    _, predictions = torch.max(outputs, 1)  
  
# Plot 10 sampel  
fig, axes = plt.subplots(2, 5, figsize=(16, 7))  
fig.suptitle('Prediksi Model pada Data Test MNIST',  
             fontsize=16, fontweight='bold', y=1.02)  
  
for i, ax in enumerate(axes.flat):  
    img = test_images[i].squeeze().cpu().numpy()  
    true_label = test_labels[i].item()  
    pred_label = predictions[i].cpu().item()  
    confidence = probabilities[i][pred_label].cpu().item() * 100  
  
    ax.imshow(img, cmap='gray')  
  
    # Warna hijau jika benar, merah jika salah  
    color = 'green' if pred_label == true_label else 'red'  
    ax.set_title(f'Pred: {pred_label} ({confidence:.1f}%) \n True:  
{true_label}',  
                fontsize=11, fontweight='bold', color=color)  
    ax.axis('off')  
  
plt.tight_layout()  
plt.show()
```

3.5.1. EVALUASI MATRIKS DAN PELAPORAN LANJUT

Blok kode kesembilan menerapkan inspeksi statistik komprehensif memakai elemen *Confusion Matrix* serta laporan metrik klasifikasi. Keseluruhan observasi pada kompartemen data uji ditebak serentak guna memetakan densitas kebenaran dan kesalahan spesifik milik masing-masing digit kategori. Metrik ditabulasikan sebagai visual peta panas (*heatmap*) berwarna yang menyoroti di area digit silang mana titik buta algoritma tersebut bersarang. Hasil pelaporan ini turut menyediakan dekonstruksi metrik ke dalam kategori *precision*, *recall*, dan indikator f1 yang murni bersifat saintifik.

CODE

```
# =====
# EVALUASI DETAIL
# =====

from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Prediksi seluruh data test
model_mnist.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model_mnist(images)
        _, predicted = torch.max(outputs, 1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

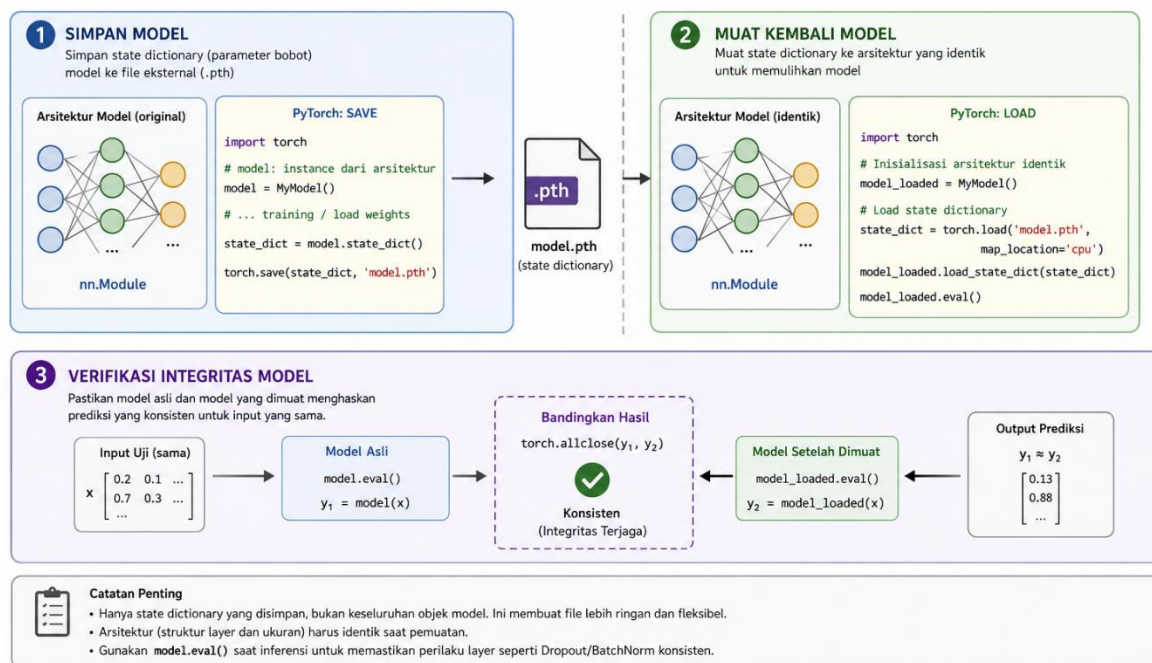
# Classification Report
print("\nClassification Report - MNIST:")
print("=" * 60)
print(classification_report(all_labels, all_preds,
                           target_names=[f'Digit {i}' for i in
range(10)]))

# Confusion Matrix
cm = confusion_matrix(all_labels, all_preds)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=[str(i) for i in range(10)],
            yticklabels=[str(i) for i in range(10)])
plt.xlabel('Prediksi', fontsize=13)
plt.ylabel('Aktual', fontsize=13)
plt.title('Confusion Matrix - MNIST', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()
```


3.5.J. PRESERVASI DAN REKONSTRUKSI MODEL (SAVING MODEL)

Blok kode kesepuluh, sebagai instrumen pamungkas, berorientasi kepada penyimpanan arsitektur usai dilatih untuk kegunaan pengaplikasian perangkat lunak masa depan (*deployment*). Format serialisasi difokuskan secara eksklusif dalam mengekstrak kerangka *state_dict*, kamus fundamental tempat segala kalkulasi pembobotan bermuara. Komponen selanjutnya mengilustrasikan mekanisme merakit ulang arsitektur struktur dari ruang kosong menuju siap pakai, lewat memanggil fungsi beban muatan. Hal ini didemonstrasikan via penyuntikan satu keping data percobaan untuk mereplikasikan performa utuh jaringan kecerdasan tanpa mengulangi pelatihan sedari pangkal.



Gambar 3.19: Prosedur persistensi model menggunakan PyTorch. Proses ini mencakup penyimpanan parameter bobot (state dictionary) ke dalam file eksternal berformat .pth dan verifikasi integritas model melalui pemuatan kembali (loading) pada arsitektur yang identik untuk memastikan konsistensi hasil prediksi.

CODE

```
# =====
# SIMPAN MODEL
# =====

# Simpan seluruh state model
torch.save(model_mnist.state_dict(), 'model_mnist_ann.pth')
print("(V) Model berhasil disimpan ke 'model_mnist_ann.pth'")

# =====
# MEMUAT MODEL (untuk penggunaan di masa depan)
# =====

# Buat arsitektur model yang sama
model_loaded = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, 10)
)

# Muat bobot
model_loaded.load_state_dict(torch.load('model_mnist_ann.pth'))
model_loaded.eval()
print("(V) Model berhasil dimuat!")

# Verifikasi
with torch.no_grad():
    sample = test_images[0:1] # Ambil 1 sampel
    output = model_loaded(sample)
    _, pred = torch.max(output, 1)
    print(f"Prediksi model yang dimuat: {pred.item()}")
    print(f"Label sebenarnya : {test_labels[0].item()}")
```